```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P08.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S08_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001ⓓ
    ///      Boot            UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P08.Boot
{
    /// &lt;summary&gt;
```

```
/// Boot     T001
/// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY_MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //              JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P08.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
```

```
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK X006                  TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
```

```
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.P08.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S08_Main";
        public const string Battle = "S08_Battle";
        public const string Explore = "S03_Explore";   // T009 case-1 exploration page
        public const string Archive = "S02_Archive";   // T009 slice-4 chapter/archive page
        public const string Unlock = "S07_Unlock";    // T015 closing slice chapter-unlock page
    }
    /// &lt;summary&gt;S08          →S08_Battle M5      ——           &lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        AboutPanel _aboutPanel;
        public void StartCase()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        /// &lt;summary&gt;T009 slice-2: the case-1 exploration entry (the
        /// CaseOneButton persistent listener ExploreSetup injects).
        /// T010 slice-4: every S03 entry caller names the case right
        /// before the load (the ExploreLauncher handoff contract).&lt;/summary&gt;
        public void StartCaseOne()
        {
            Sy.P08.Adventure.ExploreLauncher.PendingCaseId = Sy.P08.Adventure.ExploreLauncher.CaseDefault;
            SceneManager.LoadScene(NewbieSceneIds.Explore);
        }
        /// &lt;summary&gt;T010 slice-4: the case-2 exploration entry (the
        /// CaseTwoButton persistent listener CaseTwoSetup injects) -
        /// the one S03 chain loads the case-2 nine configs.&lt;/summary&gt;
        public void StartCaseTwo()
        {
            Sy.P08.Adventure.ExploreLauncher.PendingCaseId = "case02";
            SceneManager.LoadScene(NewbieSceneIds.Explore);
        }
        /// &lt;summary&gt;T009 slice-4: the chapter/archive entry (the
        /// ArchiveButton persistent listener ArchiveSetup injects).&lt;/summary&gt;
        public void StartArchive()
        {
```

```
            SceneManager.LoadScene(NewbieSceneIds.Archive);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P08_scene_01", "P08_unit_01");
            BuildAboutButton();
        }
        /// &lt;summary&gt;T030 compliance face: the about/disclosure entry on the
        /// main menu (bottom-left corner seat, programmatic UGUI - zero
        /// scene YAML, the setup-injected persistent listeners stay
        /// untouched). The label rides the cfg_about copy table.&lt;/summary&gt;
        void BuildAboutButton()
        {
            var canvas = FindObjectOfType&lt;Canvas&gt;();
            if (canvas == null) return;
            _aboutPanel = AboutPanel.AttachTo(canvas.transform);
            var btnGo = new GameObject("AboutButton", typeof(RectTransform), typeof(Image), typeof(Button));
            btnGo.transform.SetParent(canvas.transform, false);
            var rt = (RectTransform)btnGo.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0f, 0f);
            rt.anchoredPosition = new Vector2(150f, 80f);
            rt.sizeDelta = new Vector2(220f, 80f);
            var img = btnGo.GetComponent&lt;Image&gt;();
            img.color = new Color(1f, 0.702f, 0.278f, 0.92f);   // warm light #FFB347
            var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
            labelGo.transform.SetParent(btnGo.transform, false);
            var lrt = (RectTransform)labelGo.transform;
            lrt.anchorMin = Vector2.zero;
            lrt.anchorMax = Vector2.one;
            lrt.offsetMin = lrt.offsetMax = Vector2.zero;
            var label = labelGo.GetComponent&lt;Text&gt;();
            label.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            label.text = AboutPanel.Text(AboutPanel.KeyAboutButton);
            label.fontSize = 40;
            label.alignment = TextAnchor.MiddleCenter;
            label.color = new Color(0.051f, 0.051f, 0.051f);
            label.raycastTarget = false;
            btnGo.GetComponent&lt;Button&gt;().onClick.AddListener(OpenAbout);
        }
        /// &lt;summary&gt;Opens the disclosure panel (AIGC label + ad-cap + age
        /// rating - the X433 promise-book AIGC gap closed in-game).&lt;/summary&gt;
        void OpenAbout()
        {
            if (_aboutPanel != null) _aboutPanel.Open();
        }
    }
}
// AdPointConfig.cs
using System;
```

```
using UnityEngine;
namespace Sy.P08.Adventure
{
    /// cfg_ad_point data contract (T015): the chapter-economy ad point
    /// table - the task book section four V1.1 final values as pure data.
    /// Plain serializable classes so JsonUtility round-trips the whole
    /// funnel table (config-as-data face). The AdPointFunnel static face
    /// and the AdFunnelCheck gate consume this one contract - contract
    /// before data, no orphan data.
    [Serializable]
    public class AdPointTableFile
    {
        public int globalCapDaily = 12;
        public float[] gainDecayLadder = new float[0];
        public AdPriorityLadderBlock priorityLadder;
        public AdPointRow[] points = new AdPointRow[0];
    }
    [Serializable]
    public class AdPriorityLadderBlock
    {
        public int rescue;
        public int @double;   // C# keyword escape; JsonUtility maps the JSON "double" key
        public int speedup;
        public int direct;
        public int misc;
    }
    [Serializable]
    public class AdGateBlock
    {
        public int channels;            // chapter_unlock: 2 ads / 2 days / yinde-free
        public int adViewsPerUnlock;
        public string freePass = "";
        public int yindeFreeCost;       // chapter_unlock: S07 yinde channel seed cost (declared assumption in cfg meta;
        public bool chapterEndOnly;     // interstitial: sim-side chapter-end gate
    }
    [Serializable]
    public class AdPointRow
    {
        public string pointId = "";
        public string type = "";        // "rewarded" | "interstitial"
        public string @class = "";      // rescue / double / speedup / direct / misc
        public int priority;
        public int dailyCap;            // design cap, dimension = capScope (sim-side)
        public string capScope = "";    // "puzzle" | "chapter"
        public int dayCap;              // AdCapPolicy daily safety-valve pool
        public string adCategory = "";  // p08_-isolated category id (AdCapPolicy wiring)
        public int coolDownSec;
        public AdGateBlock gate;        // null when the row carries no gate block
        public string decay = "";       // empty = constant full value; "gainLadder" = decay face
        public string rewardRef = "";
```

```
        public string uiStyle = "";
    }
    /// Static load face: the shipped Resources copy is the single runtime
    /// authority (the Design/configs/P08 archive copy is the versioned
    /// source; the AdFunnelCheck gate asserts the dual faces are
    /// byte-identical, the X364 SHA256 law).
    public static class AdPointTable
    {
        static AdPointTableFile _cache;
        public static AdPointTableFile Load(bool forceReload = false)
        {
            if (_cache != null &amp;&amp; !forceReload) return _cache;
            var asset = Resources.Load&lt;TextAsset&gt;("cfg_ad_point");
            if (asset == null)
            {
                Debug.LogError("[AdPointTable] cfg_ad_point missing from Resources");
                return null;
            }
            _cache = JsonUtility.FromJson&lt;AdPointTableFile&gt;(asset.text);
            return _cache;
        }
        /// Table row for a cfg point id (null = unknown id).
        public static AdPointRow Row(string pointId)
        {
            var table = Load();
            if (table?.points == null) return null;
            foreach (var p in table.points)
                if (p != null &amp;&amp; p.pointId == pointId) return p;
            return null;
        }
    }
}
// AdPointFunnel.cs
using System.Collections.Generic;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.P08.Adventure
{
    /// T015 - the cfg_ad_point consumer face: the P02 three-piece pattern
    /// (HasCategoryBudget pre-check -&gt; category-gated consume -&gt; evt_ad
    /// funnel telemetry) as pure functions over the shared AdCapPolicy, so
    /// the S07 chapter-lock view, the wen-mi host (HintSystem), the archive
    /// bonus face and the AdFunnelCheck gate stay thin call sites.
    /// Per-puzzle and per-chapter caps stay sim-side (HintSystem W1
    /// tier==ads ladder, ArchiveBook SYS-06 one-ad-card law, the chapter
    /// lock's 2-ads-or-2-days-or-yinde-free channels); this face owns the
    /// day pools and the interstitial face. Zero server (red line 12) -
    /// policy and table are both local; a missing table row reads as
    /// "no offer" (fail-loud callers surface the missing asset through
    /// AdPointTable.Load's error log).
```

```
public static class AdPointFunnel
{
    /// Runtime ad point id: p08_-isolated namespace (P01 T020
    /// precedent) - the ads service and evt_ad telemetry never see a
    /// bare id that could collide with the P01/P02/P06/G namespaces.
    public static string AdPointId(string pointId) =&gt; "p08_" + pointId;
    /// Table row for a cfg point id (null = unknown id).
    public static AdPointRow Row(string pointId) =&gt; AdPointTable.Row(pointId);
    /// The shared policy wired exactly as the table reads: the global
    /// hard cap, the interstitial cooldown and the interstitial day
    /// valve all come from the data table (single authority).
    public static AdCapPolicy CreatePolicy(AdPointTableFile table)
    {
        int global = table != null &amp;&amp; table.globalCapDaily &gt; 0
            ? table.globalCapDaily : AdCapPolicy.DefaultDailyCap;
        float cool = AdCapPolicy.DefaultInterstitialCooldownSec;
        int interCap = 0;
        if (table != null &amp;&amp; table.points != null)
            foreach (var p in table.points)
            {
                if (p == null || p.type != "interstitial") continue;
                if (p.coolDownSec &gt; 0) cool = p.coolDownSec;
                if (p.dayCap &gt; 0) interCap = p.dayCap;
            }
        return new AdCapPolicy(global, cool, interCap);
    }
    /// Section-six pre-offer gate (P02 paradigm): true when the point's
    /// p08_ category still has day budget. The global cap is enforced at
    /// consume; per-puzzle/per-chapter caps are sim-side and owned by
    /// the host.
    public static bool PrecheckBudget(string pointId, string dayKey, AdCapPolicy policy)
    {
        var row = Row(pointId);
        if (row == null || row.type != "rewarded") return false;
        return policy.HasCategoryBudget(dayKey, AdCategories.CategoryOfPoint(AdPointId(pointId)));
    }
    /// Category-gated consume (P02 paradigm): day pool first, then the
    /// global hard cap; both counters advance together. False = the
    /// host tracks the funnel "blocked" state and never retries the ad.
    public static bool TryConsume(string pointId, string dayKey, AdCapPolicy policy)
    {
        var row = Row(pointId);
        if (row == null || row.type != "rewarded") return false;
        return policy.TryConsumeRewarded(dayKey, AdCategories.CategoryOfPoint(AdPointId(pointId)));
    }
    /// Gain decay face (yinde_boost only): gain-class points walk the
    /// ladder by same-day watch index (watch 1 = 1.0, 2 = 0.9, ...);
    /// every other P08 point keeps full value on every watch. watchIdx
    /// is 1-based; a missing table reads as full value.
    public static float GainMul(string pointId, int watchIdx)
```

```
        {
            var row = Row(pointId);
            var ladder = AdPointTable.Load()?.gainDecayLadder;
            if (row == null || ladder == null || ladder.Length == 0) return 1f;
            if (string.IsNullOrEmpty(row.decay)) return 1f;
            int idx = watchIdx &amp;lt;= 1 ? 0
                : watchIdx &amp;gt; ladder.Length ? ladder.Length - 1
                : watchIdx - 1;
            return ladder[idx];
        }
        /// Interstitial face: chapter-end only (the sim-side gate - the
        /// host never offers mid-chapter) riding the policy's cooldown +
        /// day valve (the most restrained interstitial in the portfolio).
        public static bool InterstitialAllowed(bool chapterEnd, string dayKey, float nowSec, AdCapPolicy policy)
        {
            if (!chapterEnd) return false;
            if (Row("interstitial") == null) return false;
            return policy.TryConsumeInterstitial(dayKey, nowSec);
        }
        /// evt_ad funnel telemetry (P02 vocabulary): states =
        /// offer/accept/decline/ad_fail/blocked/complete. The emitted
        /// pointId is the p08_-isolated runtime id; chapterId rides the
        /// fields so the chapter P&amp;amp;L face can read per-chapter funnels
        /// from the local log (zero server, red line 12).
        public static void Track(TelemetryService telemetry, string state,
            string pointId, string chapterId, string reason)
        {
            if (telemetry == null) return;
            var fields = new Dictionary&amp;lt;string, string&amp;gt;
            {
                { "state", state },
                { "pointId", AdPointId(pointId) },
            };
            if (chapterId != null) fields["chapterId"] = chapterId;
            if (reason != null) fields["reason"] = reason;
            telemetry.Track(EventIds.Ad, fields);
        }
    }
}
// ArchiveBook.cs
using System;
using System.Collections.Generic;
namespace Sy.P08.Adventure
{
    /// Folk archive book logic core (T009): pure C#, zero rendering -
    /// the CaseSim pattern. One book owns one chapter's collectible
    /// knowledge cards; the chapter view slice, the rewarded-ad host
    /// and the ArchiveCheck gate share this one rule set.
    ///  Laws:
    ///   A1  clue-pinned cards unlock exactly when their board clue is
```

```
///       found - repeated finds and unknown clue ids change nothing;
///   A2  the chapter's single archive_extra card unlocks only via
///       the rewarded-ad callback (GrantAdUnlock) - never via clues,
///       and clue cards never unlock via the callback;
///   A3  at most one ad card per chapter (SYS-06 archive_extra law)
///       - a grant past the cap is refused with zero state change;
///   A4  local-authority break resume: CaptureState/RestoreState
///       round-trips the unlocked set silently (no events on
///       restore); a state from another chapter is refused and the
///       book stays fresh (red line 12: zero server, bad save data
///       never blocks play).
public sealed class ArchiveBook
{
    public const int MaxAdCardsPerChapter = 1;   // SYS-06 archive_extra law
    readonly ChapterConfig _chapter;
    readonly HashSet&lt;string&gt; _unlocked = new HashSet&lt;string&gt;();
    readonly Dictionary&lt;string, List&lt;ArchiveCardDef&gt;&gt; _cluePins =
        new Dictionary&lt;string, List&lt;ArchiveCardDef&gt;&gt;();   // clueId -&gt; pinned cards
    ArchiveCardDef _adCard;                        // null when the chapter ships none
    public ArchiveBook(ChapterConfig chapter)
    {
        if (chapter == null) throw new ArgumentNullException("chapter");
        var problems = Validate(chapter);
        if (problems.Count &gt; 0)
            throw new ArgumentException("chapter config invalid: " + string.Join("; ", problems.ToArray()));
        _chapter = chapter;
        foreach (var card in chapter.archiveCards)
        {
            if (card.unlockType == "ad") { _adCard = card; continue; }
            List&lt;ArchiveCardDef&gt; pins;
            if (!_cluePins.TryGetValue(card.unlockRef, out pins))
            {
                pins = new List&lt;ArchiveCardDef&gt;();
                _cluePins[card.unlockRef] = pins;
            }
            pins.Add(card);
        }
    }
    public event Action&lt;string&gt; CardUnlocked;     // (cardId) - fires on real unlocks only
    /// The "cfg_chapter tool" half: rejects any chapter data that
    /// breaks the archive laws before the game runs (the
    /// HintSystem.Validate pattern). Returns the problem list
    /// (empty = valid).
    public static List&lt;string&gt; Validate(ChapterConfig c)
    {
        var p = new List&lt;string&gt;();
        if (c == null) { p.Add("null chapter"); return p; }
        if (string.IsNullOrEmpty(c.chapterId)) p.Add("chapterId empty");
        if (c.scenes == null || c.scenes.Count == 0) p.Add("scenes empty");
        else
```

```
        {
            var seen = new HashSet&lt;string&gt;();
            foreach (var s in c.scenes)
            {
                if (string.IsNullOrEmpty(s)) { p.Add("scene id empty"); continue; }
                if (!seen.Add(s)) p.Add("scene id dup: " + s);
            }
        }
        if (c.puzzles == null) p.Add("puzzles null");
        else
        {
            var seen = new HashSet&lt;string&gt;();
            foreach (var pz in c.puzzles)
            {
                if (string.IsNullOrEmpty(pz)) { p.Add("puzzle id empty"); continue; }
                if (!seen.Add(pz)) p.Add("puzzle id dup: " + pz);
            }
        }
        if (string.IsNullOrEmpty(c.verdict)) p.Add("verdict empty");
        if (string.IsNullOrEmpty(c.freePass)) p.Add("freePass empty");
        if (c.archiveCards == null || c.archiveCards.Count == 0) p.Add("archiveCards empty");
        else
        {
            int adCards = 0;
            var ids = new HashSet&lt;string&gt;();
            foreach (var card in c.archiveCards)
            {
                if (card == null) { p.Add("null card"); continue; }
                if (string.IsNullOrEmpty(card.cardId)) { p.Add("cardId empty"); continue; }
                if (!ids.Add(card.cardId)) p.Add("cardId dup: " + card.cardId);
                if (string.IsNullOrEmpty(card.title)) p.Add("card title empty: " + card.cardId);
                if (string.IsNullOrEmpty(card.category)) p.Add("card category empty: " + card.cardId);
                if (string.IsNullOrEmpty(card.body)) p.Add("card body empty: " + card.cardId);
                if (card.unlockType == "clue")
                {
                    if (string.IsNullOrEmpty(card.unlockRef)) p.Add("clue card ref empty: " + card.cardId);
                }
                else if (card.unlockType == "ad")
                {
                    if (!string.IsNullOrEmpty(card.unlockRef)) p.Add("ad card ref must be empty: " + card.cardId);
                    adCards++;
                }
                else p.Add("unlockType unknown: '" + card.unlockType + "' @ " + card.cardId);
            }
            if (adCards &gt; MaxAdCardsPerChapter) p.Add("ad cards over cap: " + adCards);
        }
        return p;
    }
    public ChapterConfig Chapter { get { return _chapter; } }
    public IList&lt;ArchiveCardDef&gt; Cards { get { return _chapter.archiveCards; } }
```

```
public int UnlockedCount { get { return _unlocked.Count; } }
public int TotalCount { get { return _chapter.archiveCards.Count; } }
public bool IsUnlocked(string cardId)
{
    return !string.IsNullOrEmpty(cardId) &amp;amp;&amp;amp; _unlocked.Contains(cardId);
}
/// The board-clue face (A1): the host calls this when a clue
/// lands on the verdict board. Unlocks every card pinned to
/// that clue; returns whether anything newly unlocked. Unknown
/// ids and repeated finds change nothing.
public bool NotifyClueFound(string clueId)
{
    if (string.IsNullOrEmpty(clueId)) return false;
    List&lt;ArchiveCardDef&gt; pins;
    if (!_cluePins.TryGetValue(clueId, out pins)) return false;
    bool any = false;
    foreach (var card in pins)
    {
        if (_unlocked.Add(card.cardId))
        {
            any = true;
            CardUnlocked?.Invoke(card.cardId);
        }
    }
    return any;
}
/// The archive_extra rewarded-ad callback (A2/A3): unlocks the
/// chapter's single ad card. Refused (zero state change) when
/// the chapter ships no ad card or it is already unlocked. The
/// HOST owns the ad SDK face - this core never touches it
/// (red line 12).
public bool GrantAdUnlock()
{
    if (_adCard == null) return false;
    if (!_unlocked.Add(_adCard.cardId)) return false;
    CardUnlocked?.Invoke(_adCard.cardId);
    return true;
}
/// Local-authority break resume (A4): capture the unlocked set
/// for the host's save file.
[Serializable]
public class ArchiveBookState
{
    public string chapterId = "";
    public List&lt;string&gt; unlocked = new List&lt;string&gt;();
}
public ArchiveBookState CaptureState()
{
    var st = new ArchiveBookState();
    st.chapterId = _chapter.chapterId;
```

```
            foreach (var id in _unlocked) st.unlocked.Add(id);
            return st;
        }
        /// Restore into this book: refuses foreign-chapter states; ids
        /// this chapter never authored are skipped (data-safe). Restore
        /// is silent - it never fires CardUnlocked (resume must not
        /// replay unlock toasts).
        public bool RestoreState(ArchiveBookState st)
        {
            if (st == null || st.chapterId != _chapter.chapterId) return false;
            var authored = new HashSet&lt;string&gt;();
            foreach (var card in _chapter.archiveCards) authored.Add(card.cardId);
            foreach (var id in st.unlocked)
                if (!string.IsNullOrEmpty(id) &amp;&amp; authored.Contains(id)) _unlocked.Add(id);
            return true;
        }
    }
}
// ArchiveLauncher.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P08.Adventure
{
    /// &lt;summary&gt;
    /// T009 slice-4 host for the S02 chapter/archive page (T010
    /// slice-4: case-parameterized via the PendingCaseId handoff):
    /// loads the shipped configs from Resources (the runtime double
    /// face, byte identical to Design/configs/P08 - X364 law),
    /// assembles ArchiveBook + ArchiveView and routes the view's input
    /// intents: the shipped chapter tap into the real S03 scene load
    /// (the S02 -&gt; S03 flow, requirement doc section 3), the
    /// archive_extra seat into ArchiveBook.GrantAdUnlock (the host owns
    /// the rewarded-ad callback face - the core never touches the SDK,
    /// red line 12) and the deterministic case-code share seat (the
    /// X372 promise: the settled-case code lands on this page). The
    /// morality seat shows the settled newbie baseline; the T012
    /// MoralMeter owns the live source later. Archive unlock
    /// persistence (ArchiveBook A4 capture/restore) stays core-asserted
    /// this slice - the host save wiring lands together with the
    /// chapter-progress save face (T011/T012 era), so the book and the
    /// clue board always save as one unit, never out of sync.
    /// &lt;/summary&gt;
    public sealed class ArchiveLauncher : MonoBehaviour
    {
        ArchiveBook _book;
        ArchiveView _view;
        ChapterUnlockGate _gate;   // T015 closing slice: owns the chapter-row routing
        string _secondChapterId;   // probed from cfg_chapter_case02 (null until it ships)
        string _secondFreePass;    // the second chapter's own free-pass window
        void Awake()
```

```
{
    Application.targetFrameRate = 30;
    // T010 slice-4: case-parameterized book face (the
    // PendingCaseId static handoff, the ExploreLauncher
    // pattern) - the page opens the pending case's archive;
    // the default keeps case-1 behaviour bit-identical.
    string caseId = ExploreLauncher.PendingCaseId;
    var chapter = Parse&lt;ChapterConfig&gt;("cfg_chapter_" + caseId);
    var verdict = Parse&lt;VerdictConfig&gt;("cfg_verdict_" + caseId);
    _book = new ArchiveBook(chapter);
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    _view = gameObject.AddComponent&lt;ArchiveView&gt;();
    _view.Initialize(canvas, _book);
    // T010 slice-4: the chapter row addresses shipped cases by
    // id - the second seat flips live exactly when
    // cfg_chapter_case02 ships in Resources (data-driven probe;
    // a missing config keeps the locked dead face).
    var secondAsset = Resources.Load&lt;TextAsset&gt;("cfg_chapter_case02");
    _secondChapterId = null;
    _secondFreePass = "";
    if (secondAsset != null)
    {
        var second = JsonUtility.FromJson&lt;ChapterConfig&gt;(secondAsset.text);
        if (second != null)
        {
            _secondChapterId = second.chapterId;
            _secondFreePass = second.freePass;
        }
    }
    _view.SetChapterRow(ExploreLauncher.CaseDefault, _secondChapterId, secondAsset != null);
    // T015 closing slice: the shared three-channel gate owns the
    // chapter-row routing (ad/days/yinde channels, launch free-pass).
    _gate = new ChapterUnlockGate(
        AdPointFunnel.CreatePolicy(AdPointTable.Load(true)),
        new Sy.Telemetry.TelemetryService());
    // Host-fed seats: the case code recomputes locally from the
    // shipped config (any device gets the same code - red line 12).
    _view.SetMorality(10);   // settled newbie baseline (Redline11Check A8)
    _view.SetCaseCode("     " + FinalView.CaseCode(verdict));
    _view.ChapterTapped += OnChapterTapped;
    _view.AdTapped += OnAdTapped;
    _view.SyncView();
}
void Update()
{
    if (_view == null) return;
    _view.SyncView();
}
void OnChapterTapped(string chapterId)
{
```

```
            // S02 -&gt; S03 flow (T015 closing slice): only shipped ids ever
            // reach this wire. The gate owns the routing - a free-pass or
            // unlocked chapter loads the one S03 chain exactly as before
            // (zero regression for the launch window); a locked chapter
            // diverts to the S07 unlock page with the static handoff.
            if (chapterId != ExploreLauncher.CaseDefault &amp;&amp; chapterId != _secondChapterId) return;
            int index = chapterId == ExploreLauncher.CaseDefault ? 1 : 2;
            string freePass = chapterId == ExploreLauncher.CaseDefault
                ? _book.Chapter.freePass : _secondFreePass;
            if (RouteFor(_gate, chapterId, index, freePass, UnlockLauncher.TodayKey()) == RouteUnlock)
            {
                UnlockLauncher.PendingUnlockId = chapterId;
                SceneManager.LoadScene(Sy.P08.Newbie.NewbieSceneIds.Unlock);
                return;
            }
            ExploreLauncher.PendingCaseId = chapterId;
            SceneManager.LoadScene(Sy.P08.Newbie.NewbieSceneIds.Explore);
        }
        // Routing decision (pure, UnlockPageCheck-asserted): a locked
        // chapter diverts to the S07 unlock page; free-pass and unlocked
        // chapters keep the pre-gate behaviour (zero regression).
        public const string RouteExplore = "explore";
        public const string RouteUnlock = "unlock";
        public static string RouteFor(ChapterUnlockGate gate, string chapterId, int chapterIndex, string freePass, strin
        {
            var face = gate.Evaluate(chapterId, chapterIndex, freePass, todayKey);
            return face.status == ChapterUnlockGate.StatusLocked ? RouteUnlock : RouteExplore;
        }
        void OnAdTapped()
        {
            // The host owns the rewarded-ad face: the grant stands in for
            // the ad callback (A2 single channel, the core never touches
            // the SDK).
            _book.GrantAdUnlock();
        }
        static T Parse&lt;T&gt;(string resource) where T : class
        {
            var asset = Resources.Load&lt;TextAsset&gt;(resource);
            if (asset == null) throw new System.Exception("runtime config missing: " + resource);
            var cfg = JsonUtility.FromJson&lt;T&gt;(asset.text);
            if (cfg == null) throw new System.Exception("runtime config parse failed: " + resource);
            return cfg;
        }
    }
}
// ArchiveView.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
```

```
namespace Sy.P08.Adventure
{
    /// &lt;summary&gt;
    /// T009 slice-4: the S02 chapter/archive view (polished greybox;
    /// T010 slice-4: the chapter row is host-fed and case-addressed -
    /// a shipped second case flips its seat live, a not-yet-shipped
    /// one stays a locked dead face). Pure programmatic UGUI in the
    /// FinalView pattern - every element is built in Initialize (zero
    /// scene-YAML drift). Faces:
    /// the chapter-select row (shipped chapter live, future chapter
    /// locked with its free-pass window note), the folk archive book
    /// (the five cards from cfg_chapter - unlocked cards show their
    /// verbatim title/category, locked cards stay dark), the single
    /// archive_extra rewarded-ad seat (live only while the ad card is
    /// still locked - the HOST owns the ad callback face, red line 12),
    /// the case-code share seat (the X372 promise: the settled case
    /// code lands here, host-fed, deterministic - any device recomputes
    /// it from the shipped config) and the morality seat (host-fed;
    /// the T012 MoralMeter owns the live source later). Tapping an
    /// unlocked card opens the aged-paper reading overlay - the body
    /// is typeset through the T004 VerticalTextLayout solver (pure
    /// transform, V1-V10 laws live in this face). Game text comes
    /// from the configs verbatim - the view hardcodes no case content.
    /// &lt;/summary&gt;
    public sealed class ArchiveView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        ArchiveBook _book;
        Text _titleText, _toastText, _moralityText, _codeText, _counterText, _freePassText;
        Button _chapterOneButton, _chapterTwoButton, _adButton;
        RectTransform _detailOverlay;
        Text _detailTitle, _detailCategory;
        readonly Image[] _cardFills;
        readonly Text[] _cardTexts;
        readonly Button[] _cardButtons;
        readonly Text[] _cardSeals;                // T021 red seal stamp on unlocked cards
        readonly Text[] _cardCats;                // T021 slice-2 deep layout: category line under the title
        readonly Image[] _cardChips;              // T021 slice-2: category glyph chip at the left edge
        readonly string[] _cardIds;
        string _detailBodyRaw = "";
        int _detailColumns;
        // T010 slice-4: chapter-row ids (host-fed). Defaults keep the
        // pre-slice mount bit-identical when the host never calls
        // SetChapterRow - the book's own chapter id stays the first id
        // and the second seat stays the locked dead face.
        string _chapterOneId;
        string _chapterTwoId;
        bool _chapterTwoLive;
        public event Action&lt;string&gt; ChapterTapped;   // (chapterId) -&gt; host loads the chapter scene
```

```
public event Action AdTapped;                // -&gt; host calls book.GrantAdUnlock
// Art spec sheet palette (StyleGateCheck B1 locks the same hex).
static readonly Color CurtainRed = new Color(0.545f, 0f, 0f);              // #8B0000
static readonly Color WarmYellow = new Color(1f, 0.702f, 0.278f);          // #FFB347
static readonly Color SilhouetteBlack = new Color(0.051f, 0.051f, 0.051f); // #0D0D0D
static readonly Color PaperWhite = new Color(0.961f, 0.902f, 0.784f);      // #F5E6C8
static readonly Color NetherBlue = new Color(0.106f, 0.227f, 0.294f);      // #1B3A4B
static readonly Color TalismanGold = new Color(1f, 0.839f, 0.376f);        // #FFD660
// Derived (MUST stay textually after their sources, pitfall #119).
static readonly Color SlotDark = new Color(0.12f, 0.10f, 0.08f, 0.9f);
static readonly Color ChapterLocked = new Color(0.10f, 0.08f, 0.07f, 0.85f);
const float SlotTop = 150f;          // first card slot center Y
const float SlotGap = 170f;          // slot pitch
const int BodyCharsPerColumn = 16;   // T004 vertical typesetting height
const string LockedLabel = "         ";
public ArchiveView()
{
    int n = 8;   // ceiling: cfg_chapter_case01 ships five cards; extra slots stay hidden
    _cardFills = new Image[n];
    _cardTexts = new Text[n];
    _cardButtons = new Button[n];
    _cardSeals = new Text[n];
    _cardCats = new Text[n];
    _cardChips = new Image[n];
    _cardIds = new string[n];
}
public void Initialize(Canvas canvas, ArchiveBook book)
{
    _canvas = canvas;
    _book = book;
    _chapterOneId = book.Chapter.chapterId;
    _square = MakeSquareSprite();
    BuildStaticFace();
    _book.CardUnlocked += OnCardUnlocked;
    SyncView();
}
// ——— input intents (host routes them into the cores) ———
/// &lt;summary&gt;The gate face of a chapter tap: same path the Button
/// walks, so the navigation the player uses is what gets
/// asserted. The locked future chapter carries NO listener -
/// its button is a dead face by construction.&lt;/summary&gt;
public void SimulateChapterTap(string chapterId)
{
    // T010 slice-4: ids route to their own seat; unknown or
    // locked ids are dead faces (no navigation, zero intents).
    if (chapterId == _chapterOneId)
    {
        if (_chapterOneButton != null) _chapterOneButton.onClick.Invoke();
    }
    else if (_chapterTwoLive &amp;&amp; chapterId == _chapterTwoId)
```

```
    {
        if (_chapterTwoButton != null) _chapterTwoButton.onClick.Invoke();
    }
}
public void SimulateCardTap(string cardId)
{
    for (int i = 0; i &lt; _cardIds.Length; i++)
        if (_cardIds[i] == cardId &amp;&amp; _cardButtons[i] != null)
        {
            _cardButtons[i].onClick.Invoke();
            return;
        }
}
public void SimulateAdTap()
{
    if (_adButton != null &amp;&amp; _adButton.gameObject.activeSelf) _adButton.onClick.Invoke();
}
/// &lt;summary&gt;Tap-to-close on the reading overlay: same path the
/// overlay Button walks.&lt;/summary&gt;
public void SimulateDetailTap()
{
    var btn = _detailOverlay != null ? _detailOverlay.GetComponent&lt;Button&gt;() : null;
    if (btn != null &amp;&amp; _detailOverlay.gameObject.activeSelf) btn.onClick.Invoke();
}
// ——— core event handlers ———
void OnCardUnlocked(string cardId)
{
    SetToast(string.IsNullOrEmpty(HasCard(cardId)) ? "    " : "    " + CardTitle(cardId) + " ");
    SyncView();
}
// ——— host-facing setters (launcher parity) ———
/// &lt;summary&gt;T010 slice-4: the host feeds the chapter row - both
/// shipped cases ride the one S03 chain, so the seats address
/// cases by id. The second seat flips live only when the host
/// probed its chapter config in Resources (data-driven); until
/// then it stays the locked dead face with no listener. Called
/// after Initialize; unknown ids never route.&lt;/summary&gt;
public void SetChapterRow(string firstId, string secondId, bool secondLive)
{
    if (!string.IsNullOrEmpty(firstId)) _chapterOneId = firstId;
    _chapterTwoId = secondId;
    if (_chapterTwoButton == null) return;
    _chapterTwoLive = secondLive &amp;&amp; !string.IsNullOrEmpty(secondId);
    _chapterTwoButton.interactable = _chapterTwoLive;
    _chapterTwoButton.image.color = _chapterTwoLive ? NetherBlue : ChapterLocked;
    var label = _chapterTwoButton.GetComponentInChildren&lt;Text&gt;();
    if (label != null)
        label.text = _chapterTwoLive ? "   ·            " : "   ·            ";
    _chapterTwoButton.onClick.RemoveAllListeners();
    if (_chapterTwoLive)
```

```
        _chapterTwoButton.onClick.AddListener(() =&gt; ChapterTapped?.Invoke(_chapterTwoId));
}
public void SetToast(string text) { if (_toastText != null) _toastText.text = text; }
/// &lt;summary&gt;The morality seat (S02 law): the host feeds the value;
/// the T012 MoralMeter owns the live source later.&lt;/summary&gt;
public void SetMorality(int value) { if (_moralityText != null) _moralityText.text = "   · " + value; }
/// &lt;summary&gt;The case-code share seat (X372 promise): the host feeds
/// the deterministic code string.&lt;/summary&gt;
public void SetCaseCode(string text) { if (_codeText != null) _codeText.text = text; }
// ——— gate/state queries (ChapterGateCheck face) ———
public int ShippedCardCount { get { return _book != null ? _book.TotalCount : 0; } }
public bool IsAdSeatLive
{
    get { return _adButton != null &amp;&amp; _adButton.gameObject.activeSelf; }
}
public bool IsDetailOpen { get { return _detailOverlay != null &amp;&amp; _detailOverlay.gameObject.activeSelf;
public string DetailTitleText { get { return _detailTitle != null ? _detailTitle.text : ""; } }
public string DetailCategoryText { get { return _detailCategory != null ? _detailCategory.text : ""; } }
/// &lt;summary&gt;The raw body the reading overlay carries - asserted
/// verbatim against the config (game text = cfg law).&lt;/summary&gt;
public string DetailBodyText { get { return _detailBodyRaw; } }
/// &lt;summary&gt;How many vertical columns the T004 solver produced for
/// the open card (0 when no detail is open).&lt;/summary&gt;
public int DetailColumnCount { get { return _detailColumns; } }
public string MoralityText { get { return _moralityText != null ? _moralityText.text : ""; } }
public string CaseCodeText { get { return _codeText != null ? _codeText.text : ""; } }
public string CounterText { get { return _counterText != null ? _counterText.text : ""; } }
public string ToastText { get { return _toastText != null ? _toastText.text : ""; } }
public bool IsChapterOneLive
{
    get { return _chapterOneButton != null &amp;&amp; _chapterOneButton.interactable; }
}
public bool IsChapterTwoLocked
{
    get { return _chapterTwoButton != null &amp;&amp; !_chapterTwoButton.interactable; }
}
/// &lt;summary&gt;T010 slice-4: the second seat's live face (true only
/// after the host feeds a shipped second chapter id).&lt;/summary&gt;
public bool IsChapterTwoLive
{
    get { return _chapterTwoButton != null &amp;&amp; _chapterTwoButton.interactable; }
}
public Color SlotColor(int index)
{
    return index &gt;= 0 &amp;&amp; index &lt; _cardFills.Length &amp;&amp; _cardFills[index] != null
        ? _cardFills[index].color
        : Color.clear;
}
public string SlotLabel(int index)
{
```

```
    if (index &lt; 0 || index &gt;= _cardTexts.Length || _cardTexts[index] == null) return "";
    // Gate contract kept verbatim: the unlocked slot still reads
    // "title | category" even though the face now typesets the two
    // on separate lines (deep layout slice 2).
    if (_book != null &amp;&amp; _cardCats[index] != null &amp;&amp; !string.IsNullOrEmpty(_cardIds[index])
        &amp;&amp; _book.IsUnlocked(_cardIds[index]))
        return _cardTexts[index].text + "   " + _cardCats[index].text;
    return _cardTexts[index].text;
}
// ——— card face (tap = the reading law) ———
void OnCardButtonTapped(int index)
{
    string cardId = _cardIds[index];
    if (!_book.IsUnlocked(cardId))
    {
        SetToast("     ——             ");
        return;
    }
    OpenReadingFace(cardId);
}
void OpenReadingFace(string cardId)
{
    if (string.IsNullOrEmpty(HasCard(cardId))) return;
    var card = CardOf(cardId);
    _detailTitle.text = card.title;
    _detailCategory.text = card.category;
    _detailBodyRaw = card.body;
    // T004 vertical typesetting (pure transform - V1..V10 laws):
    // columns render right-to-left, folk scroll reading order.
    var columns = VerticalTextLayout.Layout(card.body, BodyCharsPerColumn);
    _detailColumns = columns.Count;
    var leftToRight = VerticalTextLayout.RenderRightToLeft(columns);
    float colW = 72f, bodyW = leftToRight.Count * colW;
    float x0 = -bodyW * 0.5f + colW * 0.5f;
    for (int k = 0; k &lt; leftToRight.Count; k++)
    {
        var chars = leftToRight[k].ToCharArray();
        var sb = new System.Text.StringBuilder(chars.Length * 2);
        for (int c = 0; c &lt; chars.Length; c++)
        {
            if (c &gt; 0) sb.Append('\n');
            sb.Append(chars[c]);
        }
        var t = MakeText(_detailOverlay, sb.ToString(), 34,
            new Vector2(x0 + k * colW, 90f), TextAnchor.MiddleCenter, colW, 600f);
        t.gameObject.name = "VertColumn";   // per-open columns, cleared on close
        t.color = SilhouetteBlack;
    }
    _detailOverlay.gameObject.SetActive(true);
    SetToast("    " + card.title + " ");
```

```
}
void OnDetailTapped()
{
    // Clear the per-open vertical columns, then hide the overlay.
    // EditMode gates walk this same path, so the immediate-destroy
    // branch keeps batch runs from stacking deferred duplicates.
    for (int i = _detailOverlay.childCount - 1; i &gt;= 0; i--)
    {
        var child = _detailOverlay.GetChild(i);
        if (child.name == "VertColumn")
        {
            if (Application.isPlaying) UnityEngine.Object.Destroy(child.gameObject);
            else UnityEngine.Object.DestroyImmediate(child.gameObject);
        }
    }
    _detailBodyRaw = "";
    _detailColumns = 0;
    _detailOverlay.gameObject.SetActive(false);
    SyncView();
}
// ──── per-intent sync ────
/// &lt;summary&gt;Idempotent full-state refresh: card slot faces from the
/// book (unlocked warm + verbatim title/category, locked dark),
/// the in-book counter, and the single archive_extra ad seat
/// (live only while the ad card is still locked - A2/A3).&lt;/summary&gt;
public void SyncView()
{
    if (_book == null) return;
    var cards = _book.Cards;
    for (int i = 0; i &lt; _cardFills.Length; i++)
    {
        bool shown = i &lt; cards.Count;
        SetActiveIfChanged(_cardFills[i].gameObject, shown);
        if (!shown) continue;
        string id = cards[i] != null ? cards[i].cardId : "";
        _cardIds[i] = id;
        bool open = _book.IsUnlocked(id);
        _cardFills[i].color = open ? WarmYellow : SlotDark;
        // Deep layout: title rides line one, the verbatim category
        // line two, led by a glyph chip whose mark IS the
        // category's first char (content-derived, zero hardcoded
        // case text in the view).
        _cardTexts[i].text = open ? cards[i].title : LockedLabel;
        _cardTexts[i].color = open ? SilhouetteBlack : PaperWhite;
        _cardTexts[i].alignment = open ? TextAnchor.MiddleLeft : TextAnchor.MiddleCenter;
        string cat = open ? (cards[i] != null ? cards[i].category : "") : "";
        _cardCats[i].text = cat;
        bool chipOn = open &amp;&amp; cat.Length &gt; 0;
        if (_cardChips[i] != null)
        {
```

```
                SetActiveIfChanged(_cardChips[i].gameObject, chipOn);
                if (chipOn)
                {
                    var glyph = _cardChips[i].GetComponentInChildren&lt;Text&gt;();
                    if (glyph != null) glyph.text = cat.Substring(0, 1);
                }
            }
            // T021: the seal stamp marks the unlocked face (layout
            // slice 1 - paper chip + seal; full card-face art later).
            if (_cardSeals[i] != null)
                SetActiveIfChanged(_cardSeals[i].gameObject, open);
        }
        if (_counterText != null)
            _counterText.text = "    " + _book.UnlockedCount + "/" + _book.TotalCount;
        // Ad seat: only the chapter's single archive_extra card rides
        // it, and only while that card is still locked.
        bool adLive = false;
        foreach (var c in cards)
            if (c != null &amp;&amp; c.unlockType == "ad" &amp;&amp; !_book.IsUnlocked(c.cardId)) { adLive = true; b
        SetActiveIfChanged(_adButton.gameObject, adLive);
    }
    // ——— static face (built once) ———
    void BuildStaticFace()
    {
        var bg = MakeCanvasChild("Bg", SilhouetteBlack);
        bg.anchorMin = Vector2.zero;
        bg.anchorMax = Vector2.one;
        bg.anchoredPosition = Vector2.zero;
        bg.sizeDelta = Vector2.zero;
        bg.GetComponent&lt;Image&gt;().raycastTarget = false;
        // T031: nine-slice screen frame from the art catalog (X027 law) -
        // above the flat Bg base, under every interactive sibling; when
        // the catalog entry is absent the procedural Bg face stays.
        var frameArt = Sy.Core.ArtRuntime.Get("P08_ui_s02_chapter_frame");
        if (frameArt != null)
        {
            var frame = MakeCanvasChild("UiFrame", Color.white);
            frame.anchorMin = Vector2.zero;
            frame.anchorMax = Vector2.one;
            frame.anchoredPosition = Vector2.zero;
            frame.sizeDelta = Vector2.zero;
            var frameImg = frame.GetComponent&lt;Image&gt;();
            frameImg.sprite = frameArt;
            frameImg.type = Image.Type.Sliced;
            frameImg.raycastTarget = false;
        }
        _titleText = MakeText(_canvas.transform, "    ·     ", 44, new Vector2(0f, 900f), TextAnchor.MiddleCenter, 1
        _titleText.color = WarmYellow;
        _moralityText = MakeText(_canvas.transform, "   · 0", 34, new Vector2(0f, 820f), TextAnchor.MiddleCenter, 60
        _moralityText.color = PaperWhite;
```

```
// Chapter-select row: shipped chapter live, future chapter
// locked behind its free-pass window (launch face).
_chapterOneButton = MakeBar("ChapterOne", "   ·             ",
    new Vector2(0f, 690f), new Vector2(980f, 120f), NetherBlue, () =&gt; ChapterTapped?.Invoke(_chapterOneId
_chapterTwoButton = MakeBar("ChapterTwo", "   ·             ",
    new Vector2(0f, 550f), new Vector2(980f, 120f), ChapterLocked, () =&gt; { });   // locked: dead face, no
_chapterTwoButton.interactable = false;
_freePassText = MakeText(_canvas.transform, "     ·         ", 26, new Vector2(0f, 460f), TextAnchor.MiddleC
_freePassText.color = TalismanGold;
// Case-code share seat (the X372 promise: the settled-case
// code lands on this page; host-fed, deterministic).
_codeText = MakeText(_canvas.transform, "", 40, new Vector2(0f, 380f), TextAnchor.MiddleCenter, 1000f, 60f);
_codeText.color = TalismanGold;
MakeText(_canvas.transform, "    ", 36, new Vector2(-420f, 270f), TextAnchor.MiddleCenter, 160f, 50f).color
_counterText = MakeText(_canvas.transform, "    0/5", 30, new Vector2(370f, 270f), TextAnchor.MiddleCenter,
_counterText.color = PaperWhite;
for (int i = 0; i &lt; _cardFills.Length; i++)
{
    int idx = i;
    var go = new GameObject("CardSlot_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = new Vector2(0f, SlotTop - i * SlotGap);
    rt.sizeDelta = new Vector2(980f, 148f);
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = SlotDark;
    // T021 slice-2 deep layout: glyph chip at the left edge +
    // two-line typeset (title over category). Every ink band
    // stays below the +55 top strip so the C7 slot canary
    // keeps sampling pure fill, and the seal lane at +445 is
    // untouched (C7/ch1/paper canaries all bit-stable).
    var chipGo = new GameObject("CatChip", typeof(RectTransform), typeof(Image));
    chipGo.transform.SetParent(go.transform, false);
    var chipRt = (RectTransform)chipGo.transform;
    chipRt.anchorMin = chipRt.anchorMax = new Vector2(0.5f, 0.5f);
    chipRt.anchoredPosition = new Vector2(-428f, 0f);
    chipRt.sizeDelta = new Vector2(64f, 64f);
    var chip = chipGo.GetComponent&lt;Image&gt;();
    chip.sprite = _square;
    chip.color = NetherBlue;
    chip.raycastTarget = false;
    var chipGlyph = MakeText(chipGo.transform, "", 34, Vector2.zero, TextAnchor.MiddleCenter, 60f, 60f);
    chipGlyph.color = PaperWhite;
    chipGlyph.raycastTarget = false;
    chipGo.SetActive(false);
    _cardChips[i] = chip;
    var txt = MakeText(go.transform, LockedLabel, 34, new Vector2(4f, 22f), TextAnchor.MiddleCenter, 740f, 5
    txt.color = PaperWhite;
```

```
            var cat = MakeText(go.transform, "", 26, new Vector2(4f, -40f), TextAnchor.MiddleLeft, 740f, 40f);
            cat.color = CurtainRed;
            cat.raycastTarget = false;
            _cardCats[i] = cat;
            // T021: red archive seal rides the unlocked face only -
            // parked at the far right edge, clear of the center
            // canary pixel (ChapterGateCheck C7 stays bit-stable).
            var seal = MakeText(go.transform, " ", 34, new Vector2(445f, 0f), TextAnchor.MiddleCenter, 60f, 60f);
            seal.color = CurtainRed;
            seal.raycastTarget = false;
            seal.gameObject.SetActive(false);
            _cardSeals[i] = seal;
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; OnCardButtonTapped(idx));
            _cardFills[i] = img;
            _cardTexts[i] = txt;
            _cardButtons[i] = go.GetComponent&lt;Button&gt;();
        }
        // The single archive_extra rewarded-ad seat (SYS-06: one per
        // chapter; the HOST owns the ad callback face - A2).
        _adButton = MakeBar("ArchiveExtra", "     ·    ", new Vector2(0f, -700f), new Vector2(460f, 110f), NetherBlu
        _toastText = MakeText(_canvas.transform, "", 30, new Vector2(0f, -850f), TextAnchor.UpperLeft, 1030f, 120f);
        _toastText.color = PaperWhite;
        _toastText.alignment = TextAnchor.UpperLeft;
        _toastText.horizontalOverflow = HorizontalWrapMode.Wrap;
        _toastText.verticalOverflow = VerticalWrapMode.Truncate;
        // Reading overlay (aged-paper case file; tap to close).
        var rGo = new GameObject("DetailOverlay", typeof(RectTransform), typeof(Image), typeof(Button));
        rGo.transform.SetParent(_canvas.transform, false);
        _detailOverlay = (RectTransform)rGo.transform;
        _detailOverlay.anchorMin = Vector2.zero;
        _detailOverlay.anchorMax = Vector2.one;
        _detailOverlay.anchoredPosition = Vector2.zero;
        _detailOverlay.sizeDelta = Vector2.zero;
        var paper = rGo.GetComponent&lt;Image&gt;();
        paper.sprite = _square;
        paper.color = PaperWhite;
        rGo.GetComponent&lt;Button&gt;().onClick.AddListener(OnDetailTapped);
        _detailTitle = MakeText(_detailOverlay, "", 42, new Vector2(0f, 800f), TextAnchor.MiddleCenter, 990f, 70f);
        _detailTitle.color = SilhouetteBlack;
        _detailCategory = MakeText(_detailOverlay, "", 28, new Vector2(0f, 730f), TextAnchor.MiddleCenter, 990f, 50f
        _detailCategory.color = CurtainRed;
        MakeText(_detailOverlay, "   ·     ", 26, new Vector2(0f, -800f), TextAnchor.MiddleCenter, 600f, 44f).color
        rGo.SetActive(false);
    }
    string HasCard(string cardId)
    {
        foreach (var c in _book.Cards)
            if (c != null &amp;&amp; c.cardId == cardId) return c.cardId;
        return null;
    }
```

```
ArchiveCardDef CardOf(string cardId)
{
    foreach (var c in _book.Cards)
        if (c != null &amp;&amp; c.cardId == cardId) return c;
    return null;
}
string CardTitle(string cardId)
{
    foreach (var c in _book.Cards)
        if (c != null &amp;&amp; c.cardId == cardId) return c.title;
    return "";
}
// ——— small builders (FinalView tool pattern) ———
Button MakeBar(string name, string label, Vector2 pos, Vector2 size, Color fill, Action onClick)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = fill;
    var t = MakeText(go.transform, label, 36, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y);
    t.color = fill.r &gt; 0.9f &amp;&amp; fill.g &gt; 0.7f ? SilhouetteBlack : PaperWhite;
    var b = go.GetComponent&lt;Button&gt;();
    b.onClick.AddListener(() =&gt; onClick());
    return b;
}
static void SetActiveIfChanged(GameObject go, bool active)
{
    if (go.activeSelf != active) go.SetActive(active);
}
RectTransform MakeCanvasChild(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    var img = rt.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    return rt;
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
```

```
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = new Vector2(w, h);
            var t = go.GetComponent&lt;Text&gt;();
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = anchor;
            t.color = new Color(0.91f, 0.89f, 0.85f);
            t.supportRichText = true;
            return t;
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// Chapter1Controller.cs
using System;
using System.Collections.Generic;
namespace Sy.P08.Adventure
{
    /// Chapter 1 greybox orchestrator (T005 slice-2b): wires the three data
    /// contracts into one playable case flow - StoryEngine drives the beats,
    /// a per-chapter TouchAdventure drives the taps, and any granted "clue_"
    /// item auto-pins onto the VerdictSystem board (the case-file handoff).
    /// Pure C#, zero rendering (the CaseSim pattern: the later view slice and
    /// the Chapter1Check gate share this one rule set).
    ///  Laws (each pinned by a Chapter1Check case):
    ///   H1  alignment + cleanliness: scenes.Count == chapters.Count,
    ///       sceneId[i] == chapterId[i] (one scene per beat) and all three
    ///       contract Validates clean;
    ///   H2  reachability: every verdict board clue is grantable by some
    ///       scene hotzone grant or combo result (no dead-case clue);
    ///   H3  clue handoff: a granted item with the "clue_" prefix is pinned
    ///       to the board automatically (clue_ = case-file id law);
    ///   H4  tribunal gate: SetSegment / CommitVerdict are refused until the
    ///       story has reached the tribunal chapter AND the board is full;
    ///   H5  M-04 retry: a wrong verdict releases the ghost, placements
    ///       survive, progress never locks - re-place and commit again;
    ///   H6  checkpoint: CaptureState stores the story position plus the
    ///       clue board; RestoreState rebuilds them (local authority, red
    ///       line 12) and a foreign / corrupt save is rejected so the host
    ///       falls back to a fresh start.
```

```
public sealed class Chapter1Controller
{
    public const string CluePrefix = "clue_";
    readonly StoryEngine _story;
    readonly List&lt;TouchAdventure&gt; _scenes;
    readonly VerdictSystem _verdict;
    readonly string _tribunalChapterId;
    readonly List&lt;string&gt; _pinned = new List&lt;string&gt;();   // clue board mirror for the save face
    public Chapter1Controller(StoryConfig story, IList&lt;SceneConfig&gt; scenes, VerdictConfig verdict)
    {
        var problems = Validate(story, scenes, verdict);
        if (problems.Count &gt; 0)
            throw new ArgumentException("chapter 1 configs invalid: " + string.Join("; ", problems.ToArray()));
        _story = new StoryEngine(story);
        _scenes = new List&lt;TouchAdventure&gt;();
        foreach (var sc in scenes) _scenes.Add(new TouchAdventure(sc));
        _verdict = new VerdictSystem(verdict);
        _tribunalChapterId = story.chapters[story.chapters.Count - 1].chapterId;
        foreach (var sc in _scenes)
            sc.ItemGranted += OnItemGranted;
    }
    public StoryEngine Story { get { return _story; } }
    public VerdictSystem Verdict { get { return _verdict; } }
    public int CluesOnBoard { get { return _pinned.Count; } }
    /// The scene matching the current story beat (H1 keeps the indexes
    /// aligned, so the chapter index IS the scene index).
    public TouchAdventure CurrentScene { get { return _scenes[_story.ChapterIndex]; } }
    public event Action&lt;string&gt; CluePinned;   // clueId (dup pin = silent)
    /// The full alignment + cleanliness + reachability tool face (the
    /// SceneConfig.Validate pattern). Returns the problem list (empty =
    /// valid) without constructing anything.
    public static List&lt;string&gt; Validate(StoryConfig story, IList&lt;SceneConfig&gt; scenes, VerdictConfig verd
    {
        var p = new List&lt;string&gt;();
        if (story == null) { p.Add("null story"); return p; }
        p.AddRange(StoryEngine.Validate(story));
        if (verdict == null) { p.Add("null verdict"); return p; }
        p.AddRange(VerdictSystem.Validate(verdict));
        if (scenes == null || scenes.Count == 0) { p.Add("no scenes"); return p; }
        foreach (var sc in scenes) p.AddRange(TouchAdventure.Validate(sc));
        if (scenes.Count != story.chapters.Count)
        {
            p.Add("scene count " + scenes.Count + " != chapter count " + story.chapters.Count);
            return p;
        }
        for (int i = 0; i &lt; scenes.Count; i++)
            if (scenes[i] != null &amp;&amp; scenes[i].sceneId != story.chapters[i].chapterId)
                p.Add("scene/chapter misalign @" + i + ": " + scenes[i].sceneId
                    + " != " + story.chapters[i].chapterId);
        // H2 reachability: every board clue must come from some pickup or
```

```
    // combo somewhere in the chapter scenes - an ungrantable clue
    // would deadlock the case (the tribunal gate never opens).
    foreach (var clue in verdict.clues)
    {
        if (clue == null || string.IsNullOrEmpty(clue.id)) continue;
        bool grantable = false;
        foreach (var sc in scenes)
        {
            if (sc == null) continue;
            foreach (var hz in sc.hotzones)
                if (hz != null &amp;&amp; hz.grantsItemId == clue.id) { grantable = true; break; }
            if (grantable) break;
            foreach (var cb in sc.combos)
                if (cb != null &amp;&amp; cb.resultItemId == clue.id) { grantable = true; break; }
            if (grantable) break;
        }
        if (!grantable) p.Add("clue not grantable anywhere: " + clue.id);
    }
    return p;
}
// ── story face ──
public void BeginStory() { _story.Begin(); }
public void AdvanceStory() { _story.Advance(); }
// ── scene face (forwards to the current beat's TouchAdventure) ──
public HotzoneDef Tap(float x, float y) { return CurrentScene.Tap(x, y); }
public bool BeginRealmSwitch(string toRealm) { return CurrentScene.BeginRealmSwitch(toRealm); }
public void TickRealm(float dt) { CurrentScene.Tick(dt); }
public float TintAlpha { get { return CurrentScene.TintAlpha; } }
public bool Combine(params string[] itemIds) { return CurrentScene.Combine(itemIds); }
// ── tribunal face (H4 gate + H5 retry) ──
/// True once the story stands on the tribunal chapter with every
/// clue pinned - the only window where placements are accepted.
public bool TribunalOpen
{
    get
    {
        return _story.Begun
            &amp;&amp; _story.CurrentChapter.chapterId == _tribunalChapterId
            &amp;&amp; _verdict.HasAllClues;
    }
}
public bool SetSegment(string segment, string optionId)
{
    if (!TribunalOpen) return false;
    return _verdict.SetSegment(segment, optionId);
}
public VerdictSystem.CommitResult CommitVerdict()
{
    if (!TribunalOpen) return VerdictSystem.CommitResult.Refused;
    return _verdict.CommitVerdict();
```

```
}
// ── clue handoff (H3) ──
void OnItemGranted(string itemId)
{
    if (itemId == null || !itemId.StartsWith(CluePrefix, StringComparison.Ordinal)) return;
    if (KnownClue(itemId) &amp;amp;&amp;amp; _verdict.AddClue(itemId))
    {
        _pinned.Add(itemId);
        CluePinned?.Invoke(itemId);
    }
}
bool KnownClue(string clueId)
{
    foreach (var c in _verdict.Config.clues)
        if (c != null &amp;amp;&amp;amp; c.id == clueId) return true;
    return false;
}
// ── checkpoint save (H6, local authority - red line 12) ──
[Serializable]
public class SaveState
{
    public string storyId = "";
    public int chapterIndex;
    public int lineIndex;
    public string[] clues = new string[0];
}
/// Captures story position + clue board as a JsonUtility string. The
/// HOST owns persistence (the ISaveService local slot) - the
/// controller stays pure. Scene exploration state (taken hotzones,
/// scene inventory) is scene-local and replays freely on restore -
/// re-tapping a taken clue zone is a silent no-op for the board.
public string CaptureState()
{
    var s = new SaveState
    {
        storyId = _story.Config.storyId,
        chapterIndex = _story.ChapterIndex,
        lineIndex = _story.LineIndex,
        clues = _pinned.ToArray()
    };
    return UnityEngine.JsonUtility.ToJson(s);
}
/// Restores a checkpoint into this controller. Returns "" on success
/// or the rejection reason (the host then falls back to a fresh
/// BeginStory so a bad save never bricks the run).
public string RestoreState(string json)
{
    if (string.IsNullOrEmpty(json)) return "empty save";
    SaveState s;
    try { s = UnityEngine.JsonUtility.FromJson&amp;lt;SaveState&amp;gt;(json); }
```

```
        BuildSettlePanel();
        SetHint("   ……");
    }
    /// &lt;summary&gt;
    /// Deterministic presentation clock (T008 slice 2b seam): advances every
    /// beat-relevant animation by dt. Play mode: Update() feeds Time.deltaTime
    /// (same cadence the old coroutines had). Editor gates (Redline11RtCheck):
    /// fed directly, giving exact control over mid-course beat moments.
    /// Idempotent for dt=0 (asserts stay deterministic without frame time).
    /// &lt;/summary&gt;
    public void PresentTick(float dt)
    {
        _presentTime += dt;
        // T019: paper-figure tilt beat through the shared micro-motion
        // template (law unchanged: still figure until the arm window).
        _figureMotion.Tick(dt);
        _figureMotion.Apply(_paperFigure);
        // Candle breathing (0.5Hz) + realm temperature color flip.
        float breathe = 0.5f + 0.5f * Mathf.Sin(_presentTime * Mathf.PI);
        _candleGlow.localScale = Vector3.one * (0.96f + 0.08f * breathe);
        _candleImage.color = _realmCold ? CandleCold : CandleWarm;
        // Drum shake: first kick lands inside PlayDrumBeat itself, ticks decay it.
        if (_shakeActive)
        {
            _shakeT += dt;
            var rt = (RectTransform)_canvas.transform;
            if (_shakeT &gt;= ShakeSeconds)
            {
                _shakeActive = false;
                rt.anchoredPosition = _shakeOrigin;
            }
            else
            {
                rt.anchoredPosition = _shakeOrigin + (Vector2)(UnityEngine.Random.insideUnitCircle * 10f);
            }
        }
        // Realm-switch overlay: warm-&gt;cold lerp across the pinned switch face.
        if (_switchActive)
        {
            _switchT += dt;
            if (_switchT &gt;= _sim.Tuning.SwitchSeconds)
            {
                _switchActive = false;
                _switchOverlay.gameObject.SetActive(false);
            }
            else
            {
                float k = Mathf.Clamp01(_switchT / _sim.Tuning.SwitchSeconds);
                _switchImage.color = Color.Lerp(SwitchWarm, SwitchCold, k);
            }
```

```
    }
}
void Update()
{
    PresentTick(Time.deltaTime);
}
// ─────    ─────
void BuildScene()
{
    var bg = MakeCanvasChild("Bg", SilhouetteBlack);
    bg.anchorMin = Vector2.zero;
    bg.anchorMax = Vector2.one;
    bg.anchoredPosition = Vector2.zero;
    bg.sizeDelta = Vector2.zero;
    // X027 generated art: swap the solid Bg for the shadow-play stage painting.
    var bgImg = bg.GetComponent&lt;Image&gt;();
    var bgArt = Sy.Core.ArtRuntime.Get("P08_scene_01");
    if (bgArt != null)
    {
        bgImg.sprite = bgArt;
        bgImg.color = Color.white;
        bgImg.preserveAspect = true;
        bgImg.raycastTarget = false;
    }
    //
    var curtain = MakeCanvasChild("Curtain", CurtainRed);
    curtain.anchoredPosition = new Vector2(0, 820);
    curtain.sizeDelta = new Vector2(1080, 280);
    //         ──                          →
    _candleGlow = MakeCanvasChild("CandleGlow", CandleWarm);
    _candleImage = _candleGlow.GetComponent&lt;Image&gt;();
    _candleGlow.anchoredPosition = new Vector2(0, -60);
    _candleGlow.sizeDelta = new Vector2(860, 860);
    //      M1
    var lanternGo = new GameObject("Lantern", typeof(RectTransform), typeof(Image), typeof(Button));
    lanternGo.transform.SetParent(_canvas.transform, false);
    _lanternRt = (RectTransform)lanternGo.transform;
    _lanternRt.anchorMin = _lanternRt.anchorMax = new Vector2(0.5f, 0.5f);
    _lanternRt.anchoredPosition = new Vector2(-320, 220);
    _lanternRt.sizeDelta = new Vector2(150, 190);
    var lanternImg = lanternGo.GetComponent&lt;Image&gt;();
    lanternImg.sprite = _square;
    lanternImg.color = WarmYellow;
    MakeText(_lanternRt, " ", 66, Vector2.zero, TextAnchor.MiddleCenter, 150, 190).color = SilhouetteBlack;
    lanternGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; LanternTapped?.Invoke());
    //    M0     ──X027 generated art when present
    _paperFigure = MakeCanvasChild("PaperFigure", PaperWhite);
    _paperFigure.anchoredPosition = new Vector2(340, 160);
    _paperFigure.sizeDelta = new Vector2(90, 150);
    var paperImg = _paperFigure.GetComponent&lt;Image&gt;();
```

```
var paperArt = Sy.Core.ArtRuntime.Get("P08_unit_01");
if (paperArt != null)
{
    _paperFigure.sizeDelta = new Vector2(170, 280);
    paperImg.sprite = paperArt;
    paperImg.color = Color.white;
    paperImg.preserveAspect = true;
}
else
{
    paperImg.sprite = _square;
    paperImg.color = PaperWhite;
}
MakeText(_paperFigure, " ", 48, Vector2.zero, TextAnchor.MiddleCenter, 90, 150).color = SilhouetteBlack;
//      M2.5    ——"       "
var corpseGo = new GameObject("Corpse", typeof(RectTransform), typeof(Image), typeof(Button));
corpseGo.transform.SetParent(_canvas.transform, false);
_corpseRt = (RectTransform)corpseGo.transform;
_corpseRt.anchorMin = _corpseRt.anchorMax = new Vector2(0.5f, 0.5f);
_corpseRt.anchoredPosition = new Vector2(0, -60);
_corpseRt.sizeDelta = new Vector2(240, 400);
var corpseImg = corpseGo.GetComponent&lt;Image&gt;();
corpseImg.sprite = _square;
corpseImg.color = new Color(0.13f, 0.15f, 0.18f);
MakeText(_corpseRt, "  \n      ", 36, Vector2.zero, TextAnchor.MiddleCenter, 240, 400).color = PaperWhite;
corpseGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; CorpseTapped?.Invoke());
corpseGo.SetActive(false);
//      M4    ——"       "
var ghostGo = new GameObject("Ghost", typeof(RectTransform), typeof(Image), typeof(Button));
ghostGo.transform.SetParent(_canvas.transform, false);
_ghostRt = (RectTransform)ghostGo.transform;
_ghostRt.anchorMin = _ghostRt.anchorMax = new Vector2(0.5f, 0.5f);
_ghostRt.anchoredPosition = new Vector2(0, -140);
_ghostRt.sizeDelta = new Vector2(240, 240);
var ghostImg = ghostGo.GetComponent&lt;Image&gt;();
ghostImg.sprite = _square;
ghostImg.color = NetherBlue;
MakeText(_ghostRt, "  \n      ", 40, Vector2.zero, TextAnchor.MiddleCenter, 240, 240).color = PaperWhite;
ghostGo.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; GhostTapped?.Invoke());
ghostGo.SetActive(false);
//      M3      ——
var burnGo = new GameObject("BurnButton", typeof(RectTransform), typeof(Image), typeof(Button));
burnGo.transform.SetParent(_canvas.transform, false);
var burnRt = (RectTransform)burnGo.transform;
burnRt.anchorMin = burnRt.anchorMax = new Vector2(0.5f, 0f);
burnRt.anchoredPosition = new Vector2(0, 120);
burnRt.sizeDelta = new Vector2(460, 130);
var burnImg = burnGo.GetComponent&lt;Image&gt;();
burnImg.sprite = _square;
burnImg.color = CurtainRed;
```

```
        MakeText(burnRt, "   ·   ", 52, Vector2.zero, TextAnchor.MiddleCenter, 460, 130).color = PaperWhite;
        _burnButton = burnGo.GetComponent&lt;Button&gt;();
        _burnButton.interactable = true;
        _burnButton.onClick.AddListener(() =&gt; BurnTapped?.Invoke());
        burnGo.SetActive(false);
    }
    void BuildHud()
    {
        var root = _canvas.transform;
        _hintText = MakeText(root, "", 42, new Vector2(0, -840), TextAnchor.MiddleCenter, 1020, 160);
        _hintText.color = PaperWhite;
        //               /
        MakeText(root, "  ", 36, new Vector2(-440, -700), TextAnchor.MiddleCenter, 160, 50).color = WarmYellow;
        for (int i = 0; i &lt; _clueSlots.Length; i++)
        {
            var slot = MakeCanvasChild("ClueSlot_" + i, new Color(0.12f, 0.10f, 0.08f, 0.9f));
            slot.anchoredPosition = new Vector2(-440, -700 - i * 130);
            slot.sizeDelta = new Vector2(110, 110);
            _clueSlots[i] = slot.GetComponent&lt;Image&gt;();
        }
    }
    void BuildDrumPanel()
    {
        _drumPanel = MakeCanvasChild("DrumPanel", new Color(0, 0, 0, 0));
        _drumPanel.GetComponent&lt;Image&gt;().color = new Color(0, 0, 0, 0);
        _drumCountText = MakeText(_drumPanel, "", 80, new Vector2(0, 420), TextAnchor.MiddleCenter, 800, 120);
        _drumCountText.color = WarmYellow;
        for (int i = 0; i &lt; 10; i++)
        {
            int digit = i;
            var go = new GameObject("Digit_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(_canvas.transform, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0f);
            rt.anchoredPosition = new Vector2(-405 + i * 90, 300);
            rt.sizeDelta = new Vector2(80, 90);
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = CurtainRed;
            _digitImages[i] = img;
            MakeText(go.transform, i.ToString(), 44, Vector2.zero, TextAnchor.MiddleCenter, 80, 90).color = PaperWhi
            _digitButtons[i] = go.GetComponent&lt;Button&gt;();
            _digitButtons[i].onClick.AddListener(() =&gt; DigitTapped?.Invoke(digit));
        }
    }
    void BuildVerdictPanel()
    {
        _verdictPanel = MakeCanvasChild("VerdictPanel", new Color(0, 0, 0, 0));
        _verdictPanel.GetComponent&lt;Image&gt;().color = new Color(0, 0, 0, 0);
        MakeText(_verdictPanel, "   ·      ", 64, new Vector2(0, 620), TextAnchor.MiddleCenter, 1000, 100).color = P
```

```
    for (int i = 0; i &lt; NewbieConfig.VerdictOptions.Length; i++)
    {
        int index = i;
        var go = new GameObject("Verdict_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_verdictPanel, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(-250 + i * 500, 480);
        rt.sizeDelta = new Vector2(420, 150);
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = _square;
        img.color = i == NewbieConfig.VerdictCorrect ? NetherBlue : new Color(0.3f, 0.12f, 0.10f);
        MakeText(go.transform, NewbieConfig.VerdictOptions[i], 48, Vector2.zero, TextAnchor.MiddleCenter, 420, 1
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; VerdictTapped?.Invoke(index));
    }
    _verdictPanel.gameObject.SetActive(false);
}
void BuildSwitchOverlay()
{
    _switchOverlay = MakeCanvasChild("SwitchOverlay", new Color(0, 0, 0, 0));
    _switchImage = _switchOverlay.GetComponent&lt;Image&gt;();
    _switchOverlay.anchorMin = Vector2.zero;
    _switchOverlay.anchorMax = Vector2.one;
    _switchOverlay.anchoredPosition = Vector2.zero;
    _switchOverlay.sizeDelta = Vector2.zero;
    MakeText(_switchOverlay, "   ·   ", 90, new Vector2(0, 0), TextAnchor.MiddleCenter, 1000, 140).color = Paper
    _switchOverlay.gameObject.SetActive(false);
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.03f, 0.03f, 0.04f, 0.96f));
    MakeText(_settlePanel, "    ", 96, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 120).color = WarmYell
    MakeText(_settlePanel,
        "        ·   +" + _sim.Tuning.YindeGain + "\n" + NewbieConfig.ArchiveEntry +
        "\n&lt;size=42&gt;               ——         &lt;/size&gt;",
        44, new Vector2(0, 340), TextAnchor.MiddleCenter, 1020, 380).color = PaperWhite;
    MakeActionButton(_settlePanel, "    ", new Vector2(-260, -300), () =&gt; UnityEngine.SceneManagement.SceneMa
    MakeActionButton(_settlePanel, "     ", new Vector2(260, -300), () =&gt; UnityEngine.SceneManagement.SceneMa
    _settlePanel.gameObject.SetActive(false);
}
// ————      ————
public void SyncView(CaseSim sim)
{
    var phase = sim.Phase;
    SetActiveIfChanged(_lanternRt.gameObject, phase == CaseSim.CasePhase.AwaitLantern);
    SetActiveIfChanged(_corpseRt.gameObject, phase == CaseSim.CasePhase.AwaitCorpse);
    SetActiveIfChanged(_ghostRt.gameObject, phase == CaseSim.CasePhase.AwaitClues);
    SetActiveIfChanged(_burnButton.gameObject, phase == CaseSim.CasePhase.AwaitBurn);
    bool inputOk = phase == CaseSim.CasePhase.AwaitInput;
    for (int i = 0; i &lt; 10; i++)
```

```
        SetActiveIfChanged(_digitButtons[i].gameObject, inputOk);
    SetActiveIfChanged(_verdictPanel.gameObject, phase == CaseSim.CasePhase.AwaitVerdict);
}
static void SetActiveIfChanged(GameObject go, bool active)
{
    if (go.activeSelf != active) go.SetActive(active);
}
// ─────    ─────
/// &lt;summary&gt;       +  +     M2    ——       &lt;/summary&gt;
public void PlayDrumBeat(int index)
{
    _drumCountText.text = "  ×" + (index + 1);
    PlayTone(NewbieConfig.DrumToneFreq, NewbieConfig.DrumToneSeconds);
    // Shake kick lands on the beat itself (same instant the old
    // coroutine's first synchronous segment displaced the canvas);
    // PresentTick decays it for ShakeSeconds afterwards.
    _shakeActive = true;
    _shakeT = 0f;
    var rt = (RectTransform)_canvas.transform;
    _shakeOrigin = rt.anchoredPosition;
    rt.anchoredPosition = _shakeOrigin + (Vector2)(UnityEngine.Random.insideUnitCircle * 10f);
}
/// &lt;summary&gt;           &lt;/summary&gt;
public void PlayBuzz()
{
    PlayTone(NewbieConfig.BuzzToneFreq, NewbieConfig.BuzzToneSeconds);
}
/// &lt;summary&gt;     &lt;/summary&gt;
public void PlayLockOpen()
{
    PlayTone(NewbieConfig.ChimeToneFreq, NewbieConfig.ChimeToneSeconds * 0.5f);
}
/// &lt;summary&gt;M3         0.8s   →   +         &lt;/summary&gt;
public void PlayRealmSwitch()
{
    _realmCold = true;
    // Warm phase renders immediately (reversal START face); the
    // warm-&gt;cold trajectory advances in PresentTick only.
    _switchActive = true;
    _switchT = 0f;
    _switchOverlay.gameObject.SetActive(true);
    _switchImage.color = SwitchWarm;
}
/// &lt;summary&gt;             0    &lt;/summary&gt;
public void OnCorpseInspected()
{
    _clueSlots[0].color = WarmYellow;
    PlayTone(NewbieConfig.ChimeToneFreq, NewbieConfig.ChimeToneSeconds * 0.5f);
}
/// &lt;summary&gt;             1    &lt;/summary&gt;
```

```
public void OnGhostSpoken()
{
    _clueSlots[1].color = NetherBlue;
    PlayTone(NewbieConfig.ChimeToneFreq, NewbieConfig.ChimeToneSeconds);
}
/// &lt;summary&gt;          &lt;/summary&gt;
public void PlayVerdictChime()
{
    PlayTone(NewbieConfig.ChimeToneFreq, NewbieConfig.ChimeToneSeconds);
}
// ——— HUD    ———
public void SetHint(string hint) =&gt; _hintText.text = hint;
public void ShowSettle()
{
    _settlePanel.gameObject.SetActive(true);
}
// ———             +               §   ———
void PlayTone(float freq, float seconds)
{
    if (_audio == null) _audio = gameObject.AddComponent&lt;AudioSource&gt;();
    _audio.PlayOneShot(MakeTone(freq, seconds));
}
static AudioClip MakeTone(float freq, float seconds)
{
    int sr = NewbieConfig.ToneSampleRate;
    int n = Mathf.Max(1, (int)(sr * seconds));
    var data = new float[n];
    for (int i = 0; i &lt; n; i++)
    {
        float k = 1f - (float)i / n;
        data[i] = Mathf.Sin(2f * Mathf.PI * freq * i / sr) * k;
    }
    var clip = AudioClip.Create("tone", n, 1, sr, false);
    clip.SetData(data, 0);
    return clip;
}
// ———    ———
RectTransform MakeCanvasChild(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    var img = rt.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    return rt;
}
RectTransform MakeOverlayPanel(string name, Color color)
{
```

```
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = color;
        return rt;
    }
    void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
    {
        var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent&lt;Image&gt;();
        image.sprite = _square;
        image.color = CurtainRed;
        MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120).color = PaperWhite;
        go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        return t;
    }
    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
```

```
        }
    }
}
// NewbieConfig.cs
namespace Sy.P08.Newbie
{
    /// &lt;summary&gt;
    /// P08           ·           V0.2   ——            X009+IConfigService      Design/configs/P08 JSON
    ///             /        ——"               "/    2 1   /    /
    /// 06    08 M0-M5
    ///       #8B0000 ×    #FFB347 ×    #0D0D0D ×   #F5E6C8 ×    #1B3A4B            L≥25
    /// &lt;/summary&gt;
    public static class NewbieConfig
    {
        public const string GameTitle = "     ";
        public const string GameSlogan = "           ";
        // SW5 flip X366: CaseSweepCheck 5 ×8seed    = e P50 9.18s·    1.75s·    PASS
        public const float IntroSeconds = 2.6f;           // M0       +
        public const float DrumBeatInterval = 0.65f;     // M2      – –
        public const float DrumFirstDelay = 0.4f;
        public const int DrumCount = 3;                  //   =         +  =
        public const int LockAnswer = 3;
        public const float SwitchSeconds = 0.55f;        // M3            →  0.5s
        // ——             §4.2  ①                  ——
        public static readonly string[] ClueNames = { "           ", "          " };
        public const string CorpseInspectText = "               ——                        ";
        public const string GhostSpeakText = "         ——       ……      ——       ";
        // ——         §4.2  ②  1    2 1          →    =       ——
        public const int VerdictCorrect = 0;             // A
        public const string VerdictCorrectText = "            +    ——          ";
        public const string VerdictWrongGuidance = "B                    ——                        ";
        public static readonly string[] VerdictOptions = { "A     ", "B     " };
        // ——         §4.3    =      +     20   ——
        public const int YindeGain = 10;                 //        +10
        public const string ArchiveEntry = "    #1                   ——             ";
        public const float MaxTutorialSeconds = 210f;   //      ≤3.5min 06  ②
        // ——                  /    /    ——           §   ——
        public const int ToneSampleRate = 44100;
        public const float DrumToneFreq = 92f;
        public const float DrumToneSeconds = 0.28f;
        public const float BuzzToneFreq = 220f;
        public const float BuzzToneSeconds = 0.18f;
        public const float ChimeToneFreq = 660f;
        public const float ChimeToneSeconds = 0.4f;
    }
}
// NewbieTuning.cs
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
```

```
namespace Sy.P08.Newbie
{
    /// &lt;summary&gt;
    /// T008 SW-1           8       readonly struct=
    ///      Design/configs/P08/cfg_newbie_tuning.json    v1.0.0    Resources
    ///      BuiltIn()     NewbieConfig    =       build JSON      ·X032
    ///    X363   §②  v1.0  /b  (firstDelay0.4+interval0.65)/c  (intro2.6)/d  (switch0.55)/e  bcd
    /// &lt;/summary&gt;
    public readonly struct NewbieTuning
    {
        public readonly int Version;
        public readonly string Label;                // ConfigPin     "v1.0"/"b"/"c"/"d"/"e"
        public readonly float IntroSeconds;          // M0          ≥2.6s=
        public readonly float DrumFirstDelay;        // M2         ≥0.3s
        public readonly float DrumBeatInterval;      // M2
        public readonly int DrumCount;               //
        public readonly int LockAnswer;               //       =
        public readonly float SwitchSeconds;         // M3
        public readonly int YindeGain;               // M5
        public readonly float MaxTutorialSeconds;    //         06    ②≤210s
        public NewbieTuning(int version, string label, float introSeconds, float drumFirstDelay,
            float drumBeatInterval, int drumCount, int lockAnswer, float switchSeconds,
            int yindeGain, float maxTutorialSeconds)
        {
            Version = version;
            Label = label;
            IntroSeconds = introSeconds;
            DrumFirstDelay = drumFirstDelay;
            DrumBeatInterval = drumBeatInterval;
            DrumCount = drumCount;
            LockAnswer = lockAnswer;
            SwitchSeconds = switchSeconds;
            YindeGain = yindeGain;
            MaxTutorialSeconds = maxTutorialSeconds;
        }
    }
    /// &lt;summary&gt;
    ///          Live()=       ·    /Candidate(arm)=            /
    /// SetActiveForTests()=Editor       SW-2 CaseSweepCheck   ·
    /// shipped   Candidate("v1.0")      ——        JSON   SW4 shipped==winner
    /// &lt;/summary&gt;
    public static class NewbieTuningTable
    {
        //                       SW5    =        NewbieConfig
        public static NewbieTuning BuiltIn() =&gt; new NewbieTuning(
            1, "v1.0",
            NewbieConfig.IntroSeconds, NewbieConfig.DrumFirstDelay, NewbieConfig.DrumBeatInterval,
            NewbieConfig.DrumCount, NewbieConfig.LockAnswer, NewbieConfig.SwitchSeconds,
            NewbieConfig.YindeGain, NewbieConfig.MaxTutorialSeconds);
        static NewbieTuning? _testOverride;
```

```
/// &lt;summary&gt;                  build   IO JSON=          &lt;/summary&gt;
public static NewbieTuning Live() =&gt; _testOverride ?? BuiltIn();
/// &lt;summary&gt;Editor        batch          null       &lt;/summary&gt;
public static void SetActiveForTests(NewbieTuning? tuning) =&gt; _testOverride = tuning;
/// &lt;summary&gt;         Resources     "v1.0"               /      null &lt;/summary&gt;
public static NewbieTuning? Candidate(string profile)
{
    if (profile == "v1.0") return BuiltIn();
    var asset = Resources.Load&lt;TextAsset&gt;("cfg_newbie_tuning");
    if (asset == null) return null;
    var profiles = ArchiveFromJson(asset.text);
    if (profiles == null || !profiles.TryGetValue(profile, out var t)) return null;
    return t;
}
/// &lt;summary&gt;      profiles     /       ——SW-2
///      null           0 loud fail          &lt;/summary&gt;
public static Dictionary&lt;string, NewbieTuning&gt; ArchiveFromJson(string json)
{
    if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; root)) return null;
    if (!root.TryGetValue("profiles", out var node) || !(node is Dictionary&lt;string, object&gt; profiles)) ret
    var table = new Dictionary&lt;string, NewbieTuning&gt;();
    foreach (var kv in profiles)
    {
        if (!(kv.Value is Dictionary&lt;string, object&gt; p)) continue;
        string label = kv.Key;
        if (p.TryGetValue("label", out var l) &amp;&amp; l is string s &amp;&amp; !string.IsNullOrEmpty(s)) labe
        table[kv.Key] = new NewbieTuning(
            AsInt(p, "version"),
            label,
            AsFloat(p, "introSeconds"),
            AsFloat(p, "drumFirstDelay"),
            AsFloat(p, "drumBeatInterval"),
            AsInt(p, "drumCount"),
            AsInt(p, "lockAnswer"),
            AsFloat(p, "switchSeconds"),
            AsInt(p, "yindeGain"),
            AsFloat(p, "maxTutorialSeconds"));
    }
    return table;
}
static int AsInt(Dictionary&lt;string, object&gt; node, string key)
{
    if (!node.TryGetValue(key, out var v)) return 0;
    return v switch
    {
        long l =&gt; (int)l,
        double d =&gt; (int)d,
        _ =&gt; 0,
    };
}
```

```
        static float AsFloat(Dictionary&lt;string, object&gt; node, string key)
        {
            if (!node.TryGetValue(key, out var v)) return 0f;
            return v switch
            {
                long l =&gt; l,
                double d =&gt; (float)d,
                _ =&gt; 0f,
            };
        }
    }
}
// PuppetMicroMotion.cs
using UnityEngine;
namespace Sy.P08.Newbie
{
    /// &lt;summary&gt;
    /// T019 micro-motion template: one shared deterministic motion profile for the
    /// four paper-shadow jointed puppets (unit_01 paper figure + yincha / female /
    /// judge). Three channels - the uncanny head-tilt beat (always-on law for the
    /// M0 figure), an optional breath bob and an optional joint sway - all driven
    /// by the caller's deterministic presentation clock (PresentTick seam): zero
    /// sim state, zero Time.deltaTime reads, idempotent for dt = 0.
    /// The M0 paper figure runs the tilt channel ALONE: the beat's power is the
    /// perfectly still figure that suddenly tilts (uncanny-valley law, 06 script
    /// s0: first tilt &lt;= T+3.0s, Redline11RtCheck R6). Breath and sway arm the
    /// other three puppets on their story surfaces with per-puppet phase offsets
    /// so the group never moves in lockstep; keep phase within [0, 0.8]s so every
    /// puppet's first tilt still lands inside the 3.0s s0 beat window.
    /// Sequence-frame channels (blink / limb frames) attach when the multi-joint
    /// layered art lands (art doc 7.3 pending refinement); this template owns the
    /// transform law so frame variants only swap sprites, never re-time beats.
    /// &lt;/summary&gt;
    public sealed class PuppetMicroMotion
    {
        public const float TiltCycleSeconds = 2.2f;   // one tilt per cycle (validated CaseView preview)
        public const float TiltArmSeconds = 2.0f;     // still figure until the arm window opens
        public const float TiltAngle = 0.5f;          // uncanny beat: a hair of tilt
        const float BreathHz = 0.5f;                  // matches the candle glow (stage unity)
        const float BreathBobPixels = 3f;
        const float SwayHz = 0.25f;
        const float SwayAngle = 0.3f;
        readonly float _phase;
        float _wobble;                                // tilt cycle clock, original accumulation law
        bool _hasHome;
        Vector2 _home;
        public bool Breath { get; }
        public bool Sway { get; }
        public bool Tilt { get; }
        /// &lt;param name="phase"&gt;Per-puppet offset, clamped to [0, 0.8]s so the
```

```
        /// first tilt of every puppet in the group stays inside the 3.0s s0 window.&lt;/param&gt;
        public PuppetMicroMotion(bool breath, bool sway, bool tilt, float phase = 0f)
        {
            Breath = breath;
            Sway = sway;
            Tilt = tilt;
            _phase = Mathf.Clamp(phase, 0f, 0.8f);
        }
        /// &lt;summary&gt;Advance the motion clocks by dt (caller = the deterministic
        /// presentation seam; idempotent for dt = 0).&lt;/summary&gt;
        public void Tick(float dt)
        {
            _wobble += dt;
            if (_wobble &gt; TiltCycleSeconds) _wobble = 0f;
        }
        /// &lt;summary&gt;Apply the enabled channels to the puppet root. Rotation is
        /// exactly identity before the tilt arm window - the beat's stillness law
        /// the RT gate reads (R6: localRotation != identity marks the beat).&lt;/summary&gt;
        public void Apply(RectTransform puppet)
        {
            float t = _phase + _wobble;
            float tilt = Tilt &amp;&amp; _wobble &gt; TiltArmSeconds ? TiltAngle : 0f;
            float sway = Sway ? Mathf.Sin(t * ((2f * Mathf.PI) * SwayHz)) * SwayAngle : 0f;
            float angle = tilt + sway;
            puppet.localRotation = angle == 0f
                ? Quaternion.identity
                : Quaternion.Euler(0f, 0f, angle);
            if (Breath)
            {
                if (!_hasHome)
                {
                    _home = puppet.anchoredPosition;
                    _hasHome = true;
                }
                float bob = Mathf.Sin(t * ((2f * Mathf.PI) * BreathHz)) * BreathBobPixels;
                puppet.anchoredPosition = _home + new Vector2(0f, bob);
            }
        }
    }
}
// TutorialFlow.cs
using System;
namespace Sy.P08.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①  ②  ③  ④   ⑤   +
    ///      CaseSim    /    /   /    /   /    /             /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
```

```
{
    public int Step { get; private set; }
    readonly Action&lt;string&gt; _hint;
    readonly Action&lt;int&gt; _onStep;
    readonly CaseSim _sim;   // T008 SW-1
    public TutorialFlow(CaseSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
    {
        _sim = sim;
        _hint = hint ?? delegate { };
        _onStep = onStep ?? delegate { };
        sim.PhaseChanged += OnPhaseChanged;
        sim.LanternTaken += OnLanternTaken;
        sim.DrumBeat += OnDrumBeat;
        sim.WrongInput += OnWrongInput;
        sim.LockOpenedDone += OnLockOpened;
        sim.CorpseInspected += OnCorpseInspected;
        sim.RealmSwitchedDone += OnRealmSwitched;
        sim.GhostSpoken += OnGhostSpoken;
        sim.CluesComplete += OnCluesComplete;
        sim.VerdictGuidance += OnVerdictGuidance;
        sim.VerdictCorrect += OnVerdictCorrect;
        _hint("   ——            ");
    }
    void Advance(int step)
    {
        if (Step &gt;= step) return;
        Step = step;
        _onStep(step);
    }
    void OnLanternTaken()
    {
        Advance(1);
        _hint("     ——          ");
    }
    void OnDrumBeat(int index)
    {
        _hint("    " + (index + 1) + "  ——  ");
    }
    void OnWrongInput(int digit)
    {
        _hint("    ——              " + digit + "   ");
    }
    void OnLockOpened()
    {
        Advance(2);
        _hint("         ——         ");
    }
    void OnCorpseInspected()
    {
        _hint(NewbieConfig.CorpseInspectText);
```

```
        }
        void OnRealmSwitched()
        {
            Advance(3);
            _hint("    ——                    ");
        }
        void OnGhostSpoken()
        {
            _hint(NewbieConfig.GhostSpeakText);
        }
        void OnCluesComplete()
        {
            Advance(4);
            _hint("             ——          ");
        }
        void OnVerdictGuidance(int option)
        {
            _hint(NewbieConfig.VerdictWrongGuidance);
        }
        void OnVerdictCorrect()
        {
            Advance(5);
            _hint(NewbieConfig.VerdictCorrectText + "  +" + _sim.Tuning.YindeGain + " · " + NewbieConfig.ArchiveEntry);
        }
        void OnPhaseChanged(CaseSim.CasePhase phase)
        {
            switch (phase)
            {
                case CaseSim.CasePhase.AwaitLantern:
                    _hint("     ——       ");
                    break;
                case CaseSim.CasePhase.AwaitInput:
                    _hint("          ");
                    break;
                case CaseSim.CasePhase.AwaitCorpse:
                    _hint("   ——               ");
                    break;
                case CaseSim.CasePhase.AwaitBurn:
                    _hint("     ——                    ");
                    break;
                case CaseSim.CasePhase.Switching:
                    _hint("        ——         ");
                    break;
                case CaseSim.CasePhase.AwaitVerdict:
                    _hint("    ——          ");
                    break;
            }
        }
    }
}
```

```
// UIDemoView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using Sy.Core;
namespace Sy.P08
{
    /// &lt;summary&gt;
    /// U180 UIDemo: product-grade UGUI assembly proof (7-layer stack), fully
    /// procedural on top of the X027 two-state art law. Two mount paths:
    /// (a) scene Play - Awake builds its own camera + world-space canvas +
    /// EventSystem; (b) headless gate - MountOn(canvas) builds onto an
    /// existing rig canvas. All copy comes from Resources/cfg_uidemo.json
    /// (zero hardcoded text; X364 double face with Design/configs/P08).
    /// Layer stack - sibling order == render order (asserted by gate):
    ///   L0 Bg / L1 Hud / L2 Panel / L3 Popup / L4 Toast / L5 Guide / L6 Top
    /// Raycast policy is part of the product-grade contract:
    ///   Bg dead / Hud live-buttons-only / Panel live / Popup mask live
    ///   Toast dead / Guide dead / Top dead.
    /// &lt;/summary&gt;
    public sealed class UIDemoView : MonoBehaviour
    {
        // House palette tokens (AboutPanel / X373 canary anchors).
        static readonly Color CurtainRed = new Color(0.545f, 0f, 0f);
        static readonly Color WarmYellow = new Color(1f, 0.702f, 0.278f);
        static readonly Color PaperWhite = new Color(0.961f, 0.902f, 0.784f);
        static readonly Color Ink = new Color(0.13f, 0.09f, 0.10f);
        [Serializable]
        public sealed class CardEntry { public string name; public string icon; }
        [Serializable]
        public sealed class UIDemoConfig
        {
            public string screenTitle;
            public string hudChipLabel;
            public string hudHint;
            public string panelHeader;
            public string primaryLabel;
            public string secondaryLabel;
            public string popupTitle;
            public string popupBody;
            public string popupConfirm;
            public string popupCancel;
            public string toastClue;
            public string toastConfirm;
            public string toastSecondary;
            public string guideText;
            public string guideToggle;
```

```
            public string topChip;
            public List&lt;CardEntry&gt; cards;
        }
        public Canvas target;   // gate-injected canvas; null =&gt; self-built rig
        public readonly List&lt;string&gt; layerOrder = new List&lt;string&gt;();
        UIDemoConfig _cfg;
        Sprite _square, _curtain;
        Canvas _canvas;
        readonly List&lt;RectTransform&gt; _layers = new List&lt;RectTransform&gt;();
        // Gate-facing faces.
        public Image PanelFrame;
        public Button PrimaryButton, SecondaryButton;
        public List&lt;Button&gt; CardButtons = new List&lt;Button&gt;();
        public Image PopupMask;
        public GameObject PopupRoot, PopupDialog, GuideRoot;
        public Image ToastStrip;
        public Text ToastText, TopChipText;
        static readonly string[] LayerNames =
            { "L0_Bg", "L1_Hud", "L2_Panel", "L3_Popup", "L4_Toast", "L5_Guide", "L6_Top" };
        const string FrameStem = "P08_ui_s02_chapter_frame";
        // Editor-time direct-path fallback dirs (probed 2026-09-24: the
        // ArtCatalog ScriptableObject loads NULL headless - typed AND untyped
        // - while the sprites themselves load fine; the house null-safe law
        // was masking this in pixel-canary gates. Production runtime keeps
        // the catalog/Resources channel; this fallback serves editor and
        // batch contexts until the lane fixes the catalog headless load.)
        static readonly string[] EditorArtDirs = { "Share", "Icons" };
        static Sprite ResolveArt(string stem)
        {
            var s = ArtRuntime.Get(stem);
            if (s != null) return s;
#if UNITY_EDITOR
            if (!string.IsNullOrEmpty(stem))
            {
                for (int i = 0; i &lt; EditorArtDirs.Length; i++)
                {
                    string path = "Assets/_Game/Art/" + EditorArtDirs[i] + "/" + stem + ".png";
                    var byPath = UnityEditor.AssetDatabase.LoadAssetAtPath&lt;Sprite&gt;(path);
                    if (byPath == null)
                    {
                        // Some imports keep the sprite as a sub-asset next to a
                        // Texture2D main asset (probed 2026-09-24): sweep subs.
                        var subs = UnityEditor.AssetDatabase.LoadAllAssetsAtPath(path);
                        if (subs != null)
                            for (int j = 0; j &lt; subs.Length; j++)
                                if (subs[j] is Sprite sp) { byPath = sp; break; }
                    }
                    if (byPath != null) return byPath;
                }
            }
```

```
#endif
            return null;
        }
        void Awake()
        {
            // EditMode safety: AddComponent fires Awake in the editor too, and
            // the headless gate must control the mount (MountOn) - self-rig is
            // the Play-path only.
            if (Application.isPlaying &amp;&amp; _canvas == null) BuildAll();
        }
        /// &lt;summary&gt;Public entry for the headless gate (edit-mode mount onto
        /// an existing rig canvas; Awake is play-only).&lt;/summary&gt;
        public void MountOn(Canvas rigCanvas)
        {
            target = rigCanvas;
            if (_canvas == null) BuildAll();
        }
        void BuildAll()
        {
            _cfg = LoadCfg();
            _square = MakeSquareSprite();
            _curtain = MakeCurtainSprite();
            if (target != null)
            {
                _canvas = target;
            }
            else
            {
                // Play-mode self rig: camera + world-space canvas + events.
                var camGo = new GameObject("UIDemoCamera");
                camGo.transform.SetParent(transform, false);
                var cam = camGo.AddComponent&lt;Camera&gt;();
                cam.orthographic = true;
                cam.orthographicSize = 960f;
                cam.aspect = 1080f / 1920f;
                cam.nearClipPlane = 0.1f;
                cam.farClipPlane = 500f;
                cam.transform.position = new Vector3(0f, 0f, -100f);
                cam.cullingMask = 1 &lt;&lt; 5;
                cam.clearFlags = CameraClearFlags.SolidColor;
                cam.backgroundColor = new Color(0.30f, 0.30f, 0.35f);
                var cGo = new GameObject("UIDemoCanvas");
                cGo.transform.SetParent(transform, false);
                cGo.layer = 5;
                _canvas = cGo.AddComponent&lt;Canvas&gt;();
                _canvas.renderMode = RenderMode.WorldSpace;
                _canvas.worldCamera = cam;
                ((RectTransform)cGo.transform).sizeDelta = new Vector2(1080f, 1920f);
                if (FindObjectOfType&lt;EventSystem&gt;() == null)
                {
```

```
            var es = new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
            es.transform.SetParent(transform, false);
        }
    }
    for (int i = 0; i &lt; LayerNames.Length; i++)
    {
        var layer = MakeLayer(LayerNames[i]);
        _layers.Add(layer);
        layerOrder.Add(LayerNames[i]);
    }
    BuildBg();
    BuildHud();
    BuildPanel();
    BuildPopup();
    BuildToast();
    BuildGuide();
    BuildTop();
}
UIDemoConfig LoadCfg()
{
    var asset = Resources.Load&lt;TextAsset&gt;("cfg_uidemo");
    if (asset == null) throw new System.Exception("resource missing: cfg_uidemo");
    var cfg = JsonUtility.FromJson&lt;UIDemoConfig&gt;(asset.text);
    if (cfg == null) throw new System.Exception("resource parse failed: cfg_uidemo");
    return cfg;
}
RectTransform MakeLayer(string name)
{
    var go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.anchoredPosition = Vector2.zero;
    rt.sizeDelta = Vector2.zero;
    return rt;
}
void BuildBg()
{
    var img = _layers[0].gameObject.AddComponent&lt;Image&gt;();
    img.sprite = _curtain;
    img.raycastTarget = false;   // layer law: stage art never eats taps
}
void BuildHud()
{
    var strip = MakeRect(_layers[1], "HudStrip", new Vector2(0f, 1f), new Vector2(1f, 1f),
        new Vector2(0.5f, 1f), Vector2.zero, new Vector2(0f, 150f));
    var stripImg = strip.GetComponent&lt;Image&gt;();
    stripImg.sprite = _square;
    stripImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.86f);
```

```
        stripImg.raycastTarget = false;
        var title = MakeText(_layers[1], _cfg.screenTitle, 40, new Vector2(-180f, 885f),
            TextAnchor.MiddleRight, 680f, 80f);
        title.color = PaperWhite;
        // Currency chip (icon + value) - real catalog art, two-state law.
        var chip = MakeRect(_layers[1], "HudChip", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(330f, 885f), new Vector2(320f, 96f));
        var chipImg = chip.GetComponent&lt;Image&gt;();
        chipImg.sprite = _square;
        chipImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.6f);
        var chipIcon = MakeIcon(chip, _cfg.hudChipLabel == null ? null : CardIconAt(3), new Vector2(-95f, 0f), 64f);
        if (chipIcon != null) chipIcon.raycastTarget = false;
        var chipText = MakeText(chip.transform, _cfg.hudChipLabel, 30, new Vector2(35f, 0f),
            TextAnchor.MiddleLeft, 220f, 70f);
        chipText.color = WarmYellow;
        var hint = MakeText(_layers[1], _cfg.hudHint, 24, new Vector2(0f, 750f),
            TextAnchor.MiddleCenter, 900f, 50f);
        hint.color = new Color(PaperWhite.r, PaperWhite.g, PaperWhite.b, 0.55f);
        hint.raycastTarget = false;
    }
    void BuildPanel()
    {
        var frameRt = MakeRect(_layers[2], "PanelFrame", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(0f, -70f), new Vector2(1000f, 1330f));
        PanelFrame = frameRt.GetComponent&lt;Image&gt;();
        PanelFrame.sprite = ResolveArt(FrameStem);
        if (PanelFrame.sprite == null)
            PanelFrame.color = new Color(Ink.r, Ink.g, Ink.b, 0.35f); // X027: translucent placeholder, never an opa
        PanelFrame.type = PanelFrame.sprite == null ? Image.Type.Simple : Image.Type.Sliced;
        PanelFrame.raycastTarget = false;   // X451 law: frame decor is dead
        var header = MakeText(_layers[2], _cfg.panelHeader, 52, new Vector2(0f, 500f),
            TextAnchor.MiddleCenter, 900f, 90f);
        header.color = WarmYellow;
        // 2x2 clue cards on the panel body.
        Vector2[] pos = { new Vector2(-250f, 250f), new Vector2(250f, 250f),
                          new Vector2(-250f, -30f), new Vector2(250f, -30f) };
        for (int i = 0; i &lt; pos.Length &amp;&amp; i &lt; _cfg.cards.Count; i++)
        {
            var card = _cfg.cards[i];
            var rt = MakeRect(_layers[2], "Card_" + i, Vector2.one * 0.5f, Vector2.one * 0.5f,
                Vector2.one * 0.5f, pos[i] + new Vector2(0f, -70f), new Vector2(460f, 270f));
            var img = rt.GetComponent&lt;Image&gt;();
            img.sprite = _square;
            img.color = new Color(Ink.r, Ink.g, Ink.b, 0.55f);
            var icon = MakeIcon(rt.transform, card.icon, new Vector2(-130f, 25f), 150f);
            if (icon != null) icon.raycastTarget = false;
            var label = MakeText(rt.transform, card.name, 34, new Vector2(75f, -75f),
                TextAnchor.MiddleLeft, 330f, 80f);
            label.color = PaperWhite;
            var btn = rt.gameObject.AddComponent&lt;Button&gt;();
```

```
            int idx = i;
            btn.onClick.AddListener(delegate { ShowToast(_cfg.toastClue + " · " + card.name); MarkCard(idx); });
            CardButtons.Add(btn);
        }
        // Bottom action bar: primary (unique warm) + secondary (neutral).
        var prim = MakeRect(_layers[2], "PrimaryButton", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(-260f, -520f), new Vector2(450f, 130f));
        var primImg = prim.GetComponent&lt;Image&gt;();
        primImg.sprite = _square;
        primImg.color = WarmYellow;
        PrimaryButton = prim.gameObject.AddComponent&lt;Button&gt;();
        var primLabel = MakeText(prim.transform, _cfg.primaryLabel, 44, Vector2.zero,
            TextAnchor.MiddleCenter, 430f, 110f);
        primLabel.color = Ink;
        primLabel.raycastTarget = false;
        PrimaryButton.onClick.AddListener(OpenPopup);
        var sec = MakeRect(_layers[2], "SecondaryButton", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(260f, -520f), new Vector2(450f, 130f));
        var secImg = sec.GetComponent&lt;Image&gt;();
        secImg.sprite = _square;
        secImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.65f);
        SecondaryButton = sec.gameObject.AddComponent&lt;Button&gt;();
        var secLabel = MakeText(sec.transform, _cfg.secondaryLabel, 40, Vector2.zero,
            TextAnchor.MiddleCenter, 430f, 110f);
        secLabel.color = PaperWhite;
        secLabel.raycastTarget = false;
        SecondaryButton.onClick.AddListener(delegate { ShowToast(_cfg.toastSecondary); });
    }
    void BuildPopup()
    {
        PopupRoot = _layers[3].gameObject;
        PopupRoot.SetActive(false);
        PopupMask = _layers[3].gameObject.AddComponent&lt;Image&gt;();
        PopupMask.sprite = _square;
        PopupMask.color = new Color(0f, 0f, 0f, 0.55f);
        PopupMask.raycastTarget = true;   // layer law: modal mask blocks
        var dlg = MakeRect(_layers[3], "PopupDialog", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(0f, 30f), new Vector2(900f, 560f));
        PopupDialog = dlg.gameObject;
        var dlgImg = dlg.GetComponent&lt;Image&gt;();
        dlgImg.sprite = ResolveArt(FrameStem);
        if (dlgImg.sprite == null)
            dlgImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.55f); // placeholder law, see PanelFrame
        dlgImg.type = dlgImg.sprite == null ? Image.Type.Simple : Image.Type.Sliced;
        dlgImg.raycastTarget = false;
        var title = MakeText(dlg.transform, _cfg.popupTitle, 48, new Vector2(0f, 190f),
            TextAnchor.MiddleCenter, 820f, 90f);
        title.color = WarmYellow;
        var body = MakeText(dlg.transform, _cfg.popupBody, 38, new Vector2(0f, 40f),
            TextAnchor.MiddleCenter, 780f, 160f);
```

```
        body.color = PaperWhite;
        var ok = MakeRect(dlg, "PopupConfirm", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(-200f, -180f), new Vector2(340f, 120f));
        var okImg = ok.GetComponent&lt;Image&gt;();
        okImg.sprite = _square;
        okImg.color = WarmYellow;
        var okBtn = ok.gameObject.AddComponent&lt;Button&gt;();
        var okLabel = MakeText(ok.transform, _cfg.popupConfirm, 40, Vector2.zero,
            TextAnchor.MiddleCenter, 320f, 100f);
        okLabel.color = Ink;
        okLabel.raycastTarget = false;
        okBtn.onClick.AddListener(delegate { ClosePopup(); ShowToast(_cfg.toastConfirm); });
        var cancel = MakeRect(dlg, "PopupCancel", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(200f, -180f), new Vector2(340f, 120f));
        var cancelImg = cancel.GetComponent&lt;Image&gt;();
        cancelImg.sprite = _square;
        cancelImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.65f);
        var cancelBtn = cancel.gameObject.AddComponent&lt;Button&gt;();
        var cancelLabel = MakeText(cancel.transform, _cfg.popupCancel, 40, Vector2.zero,
            TextAnchor.MiddleCenter, 320f, 100f);
        cancelLabel.color = PaperWhite;
        cancelLabel.raycastTarget = false;
        cancelBtn.onClick.AddListener(ClosePopup);
    }
    void BuildToast()
    {
        var strip = MakeRect(_layers[4], "ToastStrip", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(0f, 700f), new Vector2(900f, 120f));
        ToastStrip = strip.GetComponent&lt;Image&gt;();
        ToastStrip.sprite = _square;
        ToastStrip.color = new Color(Ink.r, Ink.g, Ink.b, 0.88f);
        ToastStrip.raycastTarget = false;
        ToastText = MakeText(strip.transform, "", 34, Vector2.zero,
            TextAnchor.MiddleCenter, 860f, 100f);
        ToastText.color = PaperWhite;
        ToastText.raycastTarget = false;
        strip.gameObject.SetActive(false);
    }
    void BuildGuide()
    {
        GuideRoot = _layers[5].gameObject;
        GuideRoot.SetActive(false);
        var ring = MakeRect(_layers[5], "GuideRing", Vector2.one * 0.5f, Vector2.one * 0.5f,
            Vector2.one * 0.5f, new Vector2(-260f, -590f), new Vector2(560f, 230f));
        var ringImg = ring.GetComponent&lt;Image&gt;();
        ringImg.sprite = _square;
        ringImg.color = new Color(WarmYellow.r, WarmYellow.g, WarmYellow.b, 0.30f);
        ringImg.raycastTarget = false;   // demo guide: non-blocking
        var label = MakeText(_layers[5], _cfg.guideText, 40, new Vector2(140f, -590f),
            TextAnchor.MiddleLeft, 500f, 90f);
```

```
    label.color = WarmYellow;
    label.raycastTarget = false;
}
void BuildTop()
{
    TopChipText = MakeText(_layers[6], _cfg.topChip, 22, new Vector2(-290f, 790f),
        TextAnchor.MiddleLeft, 500f, 50f);
    TopChipText.color = new Color(PaperWhite.r, PaperWhite.g, PaperWhite.b, 0.6f);
    TopChipText.raycastTarget = false;
    // Top-layer toggle: flips the guide layer from anywhere.
    var tog = MakeRect(_layers[6], "GuideToggle", Vector2.one * 0.5f, Vector2.one * 0.5f,
        Vector2.one * 0.5f, new Vector2(455f, 790f), new Vector2(170f, 70f));
    var togImg = tog.GetComponent&lt;Image&gt;();
    togImg.sprite = _square;
    togImg.color = new Color(Ink.r, Ink.g, Ink.b, 0.7f);
    var togBtn = tog.gameObject.AddComponent&lt;Button&gt;();
    var togLabel = MakeText(tog.transform, _cfg.guideToggle, 26, Vector2.zero,
        TextAnchor.MiddleCenter, 160f, 60f);
    togLabel.color = PaperWhite;
    togLabel.raycastTarget = false;
    togBtn.onClick.AddListener(delegate { ToggleGuide(!GuideRoot.activeSelf); });
}
// ——— interactive faces (gate taps the same paths the player taps) ———
public void OpenPopup()
{
    PopupRoot.SetActive(true);
    if (Application.isPlaying)
    {
        StopAllCoroutines();
        StartCoroutine(PopupScaleIn());
    }
}
public void ClosePopup() { PopupRoot.SetActive(false); }
public void ShowToast(string message)
{
    ToastText.text = message;
    ToastStrip.gameObject.SetActive(true);
    if (Application.isPlaying) StartCoroutine(ToastFade());
}
/// &lt;summary&gt;Config-sourced toast (gate path: ASCII law keeps CJK out
/// of C# sources, so the demo copy always comes from cfg_uidemo).&lt;/summary&gt;
public void ShowClueToast() { ShowToast(_cfg.toastClue); }
/// &lt;summary&gt;Editor gates live in a predefined editor assembly that
/// cannot see the Sy.Core package, so art-chain diagnostics route
/// through this runtime face. Probes both load paths because the
/// Resources route has been observed to fail headless.&lt;/summary&gt;
public string ArtDiag()
{
    var cat = ArtRuntime.Catalog;
    string res = cat == null ? "resNULL" : "resEntries=" + cat.entries.Count
```

```
                + " resGet=" + (cat.Get(FrameStem) != null);
            string ad = "n/a";
#if UNITY_EDITOR
            var so = UnityEditor.AssetDatabase.LoadAssetAtPath&lt;ArtCatalog&gt;(
                "Assets/_Game/Resources/ArtCatalog.asset");
            ad = so == null ? "adNULL" : "adEntries=" + so.entries.Count
                + " adGet=" + (so.Get(FrameStem) != null);
            var untyped = UnityEditor.AssetDatabase.LoadAssetAtPath&lt;UnityEngine.Object&gt;(
                "Assets/_Game/Resources/ArtCatalog.asset");
            string untypedInfo = untyped == null ? "untypedNULL" : "untypedType=" + untyped.GetType().Name;
            var directSprite = UnityEditor.AssetDatabase.LoadAssetAtPath&lt;Sprite&gt;(
                "Assets/_Game/Art/Share/P08_ui_s02_chapter_frame.png");
            string spriteInfo = directSprite == null ? "spriteNULL"
                : "spriteOK border=" + (int)directSprite.border.x + "," + (int)directSprite.border.y
                + "," + (int)directSprite.border.z + "," + (int)directSprite.border.w;
            var iconMain = UnityEditor.AssetDatabase.LoadAssetAtPath&lt;UnityEngine.Object&gt;(
                "Assets/_Game/Art/Icons/P08_hot_s08_paper_money.png");
            string iconMainInfo = iconMain == null ? "iconMainNULL" : "iconMain=" + iconMain.GetType().Name
                + " asSprite=" + (iconMain as Sprite != null);
            var iconAll = UnityEditor.AssetDatabase.LoadAllAssetsAtPath(
                "Assets/_Game/Art/Icons/P08_hot_s08_paper_money.png");
            int spriteSubs = 0;
            if (iconAll != null)
                for (int i = 0; i &lt; iconAll.Length; i++)
                    if (iconAll[i] is Sprite) spriteSubs++;
            ad = ad + " | " + spriteInfo + " | " + iconMainInfo + " | iconSubs=" + spriteSubs;
#endif
            return res + " | " + ad;
        }
        public void ToggleGuide(bool on) { GuideRoot.SetActive(on); }
        void MarkCard(int index)
        {
            if (index &gt;= 0 &amp;&amp; index &lt; CardButtons.Count &amp;&amp; CardButtons[index] != null)
                CardButtons[index].image.color = new Color(WarmYellow.r, WarmYellow.g, WarmYellow.b, 0.35f);
        }
        IEnumerator PopupScaleIn()
        {
            var rt = (RectTransform)PopupDialog.transform;
            var targetScale = Vector3.one;
            float t = 0f;
            while (t &lt; 0.12f)
            {
                t += Time.unscaledDeltaTime;
                rt.localScale = Vector3.Lerp(new Vector3(0.92f, 0.92f, 1f), targetScale, t / 0.12f);
                yield return null;
            }
            rt.localScale = targetScale;
        }
        IEnumerator ToastFade()
        {
```

```
        float hold = 1.6f, fade = 0.4f, t = 0f;
        while (t &lt; hold) { t += Time.unscaledDeltaTime; yield return null; }
        var cg = ToastStrip.GetComponent&lt;CanvasGroup&gt;();
        if (cg == null) cg = ToastStrip.gameObject.AddComponent&lt;CanvasGroup&gt;();
        while (t &lt; hold + fade)
        {
            t += Time.unscaledDeltaTime;
            cg.alpha = 1f - (t - hold) / fade;
            yield return null;
        }
        cg.alpha = 1f;
        ToastStrip.gameObject.SetActive(false);
    }
    string CardIconAt(int index)
    {
        return _cfg != null &amp;&amp; _cfg.cards != null &amp;&amp; index &gt;= 0 &amp;&amp; index &lt; _cfg.cards.
            ? _cfg.cards[index].icon : null;
    }
    // ——— house-style static builders ———
    static RectTransform MakeRect(RectTransform parent, string name, Vector2 aMin, Vector2 aMax,
        Vector2 pivot, Vector2 pos, Vector2 size)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = aMin;
        rt.anchorMax = aMax;
        rt.pivot = pivot;
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        return rt;
    }
    static Text MakeText(Transform parent, string text, int size, Vector2 pos,
        TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        return t;
    }
```

```
static Image MakeIcon(Transform parent, string stem, Vector2 pos, float size)
{
    if (string.IsNullOrEmpty(stem)) return null;
    var sprite = ResolveArt(stem);
    if (sprite == null) return null;   // X027: missing art stays procedural
    var go = new GameObject("Icon_" + stem, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(size, size);
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = sprite;
    img.preserveAspect = true;
    return img;
}
/// &lt;summary&gt;Deterministic curtain gradient (ExploreView MakeCurtainSprite
/// profile, trimmed: red field + fold stripes + top valance).&lt;/summary&gt;
static Sprite MakeCurtainSprite()
{
    const int W = 108, H = 192;
    var tex = new Texture2D(W, H, TextureFormat.RGBA32, false);
    tex.wrapMode = TextureWrapMode.Clamp;
    var px = new Color32[W * H];
    for (int y = 0; y &lt; H; y++)
    {
        float v = y / (float)(H - 1);
        for (int x = 0; x &lt; W; x++)
        {
            float u = x / (float)(W - 1);
            float fold = 0.5f + 0.5f * Mathf.Sin(u * Mathf.PI * 9f);
            float r = CurtainRed.r * (0.72f + 0.28f * fold);
            float g = CurtainRed.g * (0.72f + 0.28f * fold);
            float b = CurtainRed.b * (0.72f + 0.28f * fold);
            float a = 0.45f + 0.25f * v;
            float valance = Mathf.Clamp01((v - 0.855f) / 0.045f);
            a = Mathf.Max(a, valance * 0.92f);
            float apron = Mathf.Clamp01((0.10f - v) / 0.06f);
            a = Mathf.Max(a, apron * 0.55f);
            px[y * W + x] = new Color32(
                (byte)Mathf.RoundToInt(r * 255f),
                (byte)Mathf.RoundToInt(g * 255f),
                (byte)Mathf.RoundToInt(b * 255f),
                (byte)Mathf.RoundToInt(a * 255f));
        }
    }
    tex.SetPixels32(px);
    tex.Apply(false, false);
    return Sprite.Create(tex, new Rect(0, 0, W, H), new Vector2(0.5f, 0.5f), W);
}
```

```
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// UiInputChainTests.cs
using System.Collections;
using NUnit.Framework;
using Sy.P08.Adventure;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem;
using UnityEngine.InputSystem.UI;
using UnityEngine.UI;
using UnityEngine.TestTools;
namespace Sy.P08.Tests
{
    /// &lt;summary&gt;
    /// A0 slice-1 (U262 batch A, the 12th doc input-chain truth law): the
    /// three core P08 flows driven by SYNTHESIZED touch input through the
    /// real UGUI pointer chain - EventSystem + InputSystemUIInputModule +
    /// GraphicRaycaster + Button state - instead of onClick.Invoke() seam
    /// calls. The rig is pure programmatic UGUI in the X460 RT-gate
    /// pattern (WorldSpace canvas + orthographic camera framing the full
    /// 1080x1920 design face): batchmode Game View sizes vary, so the
    /// overlay route would crop the upper seats off-canvas on small
    /// screens - the world-space rig keeps every seat tappable at any
    /// screen size. Each test builds its own EventSystem carrying the
    /// InputSystem module, then taps real seats at their runtime screen
    /// points. Flows: chapter tap (S02 routing intent), archive card
    /// (reading overlay open/close), unlock enter (S07 gate seat
    /// liveness). Faces are asserted against the shipped config verbatim
    /// (no case text is hardcoded here).
    /// &lt;/summary&gt;
    [TestFixture]
    public sealed class UiInputChainTests : InputTestFixture
    {
        GameObject _root;
        Canvas _canvas;
        Camera _cam;
        ChapterConfig _chapter;
        ArchiveBook _book;
        ArchiveView _archiveView;
        UnlockView _unlockView;
```

```
public override void Setup()
{
    base.Setup();
    _root = new GameObject("UiInputChainRig");
    var esGo = new GameObject("TestEventSystem");
    esGo.transform.SetParent(_root.transform, false);
    esGo.AddComponent&lt;EventSystem&gt;();
    esGo.AddComponent&lt;InputSystemUIInputModule&gt;();
    var camGo = new GameObject("TestCamera");
    camGo.transform.SetParent(_root.transform, false);
    _cam = camGo.AddComponent&lt;Camera&gt;();
    _cam.orthographic = true;
    _cam.orthographicSize = 960f;   // half the 1920 design height
    _cam.transform.position = new Vector3(0f, 0f, -1000f);
    _cam.farClipPlane = 2000f;
    var canvasGo = new GameObject("TestCanvas");
    canvasGo.transform.SetParent(_root.transform, false);
    _canvas = canvasGo.AddComponent&lt;Canvas&gt;();
    _canvas.renderMode = RenderMode.WorldSpace;
    _canvas.worldCamera = _cam;
    var canvasRt = canvasGo.GetComponent&lt;RectTransform&gt;();
    canvasRt.sizeDelta = new Vector2(1080f, 1920f);
    canvasGo.AddComponent&lt;GraphicRaycaster&gt;();
}
public override void TearDown()
{
    if (_root != null) Object.DestroyImmediate(_root);
    _root = null;
    _canvas = null;
    _cam = null;
    _chapter = null;
    _book = null;
    _archiveView = null;
    _unlockView = null;
    base.TearDown();
}
// ——— flow 1: chapter tap routes through the pointer chain ———
[UnityTest]
public IEnumerator ChapterFlow_InjectedTouch_WalksRealPointerChain()
{
    BuildArchiveFace();
    string tappedId = null;
    _archiveView.ChapterTapped += id =&gt; tappedId = id;
    // The shipped live seat: a synthesized touch at the seat's
    // runtime screen point fires the navigation intent with the
    // shipped chapter id - the same walk the player's finger takes.
    Assert.IsTrue(_archiveView.IsChapterOneLive);
    yield return TouchTap(FindRect("ChapterOne"));
    Assert.AreEqual(_chapter.chapterId, tappedId);
    // The locked future chapter stays a dead face through the
```

```
    // same chain: non-interactable seats swallow the pointer.
    tappedId = null;
    Assert.IsTrue(_archiveView.IsChapterTwoLocked);
    yield return TouchTap(FindRect("ChapterTwo"));
    Assert.IsNull(tappedId);
}
// ——— flow 2: archive card opens/closes the reading overlay ———
[UnityTest]
public IEnumerator ArchiveCardFlow_InjectedTouch_OpensAndClosesReadingOverlay()
{
    BuildArchiveFace();
    // Cards ship clue-gated: unlock card 0 the way the explore
    // loop does, then tap its slot through the real chain.
    var card = _book.Cards[0];
    Assert.IsFalse(_book.IsUnlocked(card.cardId));
    Assert.IsTrue(_book.NotifyClueFound(card.unlockRef));
    yield return TouchTap(FindRect("CardSlot_0"));
    Assert.IsTrue(_archiveView.IsDetailOpen);
    Assert.AreEqual(card.title, _archiveView.DetailTitleText);
    Assert.AreEqual(card.category, _archiveView.DetailCategoryText);
    Assert.Greater(_archiveView.DetailColumnCount, 0);
    // Tap the aged-paper overlay itself to close (same chain).
    yield return TouchTap(FindRect("DetailOverlay"));
    Assert.IsFalse(_archiveView.IsDetailOpen);
}
// ——— flow 3: unlock-enter seat fires only while it is live ———
[UnityTest]
public IEnumerator UnlockEnterFlow_InjectedTouch_FiresOnlyWhileSeatLive()
{
    BuildUnlockFace(ChapterUnlockGate.StatusFreePass);
    bool entered = false;
    _unlockView.EnterTapped += () =&gt; entered = true;
    // Free-pass face: the enter seat is live and the injected
    // tap routes it.
    Assert.IsTrue(_unlockView.IsEnterLive);
    yield return TouchTap(FindRect("EnterChapter"));
    Assert.IsTrue(entered);
    // Locked face: the enter seat hides - a touch at the old
    // rect hits nothing interactive and the intent stays quiet.
    _unlockView.SyncFace(MakeFace(ChapterUnlockGate.StatusLocked));
    Assert.IsFalse(_unlockView.IsEnterLive);
    entered = false;
    yield return TouchTap(FindRect("EnterChapter"));
    Assert.IsFalse(entered);
}
// ——— rig helpers ———
void BuildArchiveFace()
{
    var asset = Resources.Load&lt;TextAsset&gt;("cfg_chapter_case01");
    Assert.NotNull(asset, "runtime config missing: cfg_chapter_case01");
```

```
            _chapter = JsonUtility.FromJson&lt;ChapterConfig&gt;(asset.text);
            Assert.NotNull(_chapter);
            _book = new ArchiveBook(_chapter);
            var host = new GameObject("ArchiveViewHost");
            host.transform.SetParent(_root.transform, false);
            _archiveView = host.AddComponent&lt;ArchiveView&gt;();
            _archiveView.Initialize(_canvas, _book);
        }
        void BuildUnlockFace(string status)
        {
            var host = new GameObject("UnlockViewHost");
            host.transform.SetParent(_root.transform, false);
            _unlockView = host.AddComponent&lt;UnlockView&gt;();
            _unlockView.Initialize(_canvas, "case02");
            _unlockView.SyncFace(MakeFace(status));
        }
        static ChapterUnlockGate.Face MakeFace(string status)
        {
            return new ChapterUnlockGate.Face
            {
                status = status,
                adsWatched = 0,
                adsPerUnlock = 2,
                daysRemaining = 2,
                yindeCost = 30,
            };
        }
        RectTransform FindRect(string childName)
        {
            var t = _canvas.transform.Find(childName);
            Assert.NotNull(t, "canvas child missing: " + childName);
            return (RectTransform)t;
        }
        /// &lt;summary&gt;One synthesized finger tap at the target's runtime
        /// screen point: touch down, one frame, touch up, then two frames
        /// so the pointer chain (module update -&gt; EventSystem -&gt; onClick)
        /// settles before the caller asserts.&lt;/summary&gt;
        IEnumerator TouchTap(RectTransform target)
        {
            var screenPos = RectTransformUtility.WorldToScreenPoint(_cam, target.position);
            Assert.Greater(Screen.width, 0, "player has no screen face in this run");
            Assert.Greater(Screen.height, 0, "player has no screen face in this run");
            BeginTouch(1, screenPos);
            yield return null;
            EndTouch(1, screenPos);
            yield return null;
            yield return null;
        }
    }
}
```